

Networking for DevOps

From your first ping to VPCs, Docker and Kubernetes networking.

Beginner-friendly. Straight to the point. Real, relatable examples.

CREATED BY

 **Sharad Verma** [@sharadverma638](#)

IP & SUBNETTING

OSI & TCP/IP

DNS & ROUTING

CLOUD & K8S NET

16
sections

40+
illustrations

A-Z
full coverage



Want to learn DevOps?



github.com/sharadverma638/sv-opslibrary

Contents

16 sections, start to finish. Follow in order, or jump to what you need.

01	What Is the Internet? Cables, satellites, tiers and how it all connects	2
02	Data Centers & The Cloud What the cloud really is, and how AWS works	5
03	Networking Basics What a network is and the types of networks	9
04	Network Devices Switch, router, modem, hub and friends	12
05	Protocols, Packets & MAC The rules and the envelopes of the internet	13
06	IP Addressing Every device needs an address	15
07	Subnetting & CIDR Cutting big networks into small ones	19
08	The OSI Model Seven layers, one message	22
09	The TCP/IP Model The model the real internet uses	25
10	Ports & Protocols HTTP, HTTPS, mail, FTP, SSH, TCP, UDP	26
11	Routing, NAT & ARP How packets find their way	31
12	DNS The phonebook of the internet	33
13	DHCP & Network Services DHCP, firewall, load balancer, VPN, VLAN	35
14	Cloud & Container Networking VPC, Docker and Kubernetes networking	37
15	Troubleshooting Tools ping, traceroute, curl, dig and more	40
16	Cheat Sheet & Glossary Revise fast, learn next	43

01

What Is the Internet?

Cables, satellites, tiers and how it all connects

Before diving into protocols and addresses, let's answer the big question in plain language: **what actually is the Internet, and how does a message reach someone on the other side of the planet?**

A standalone machine vs a networked machine

A computer on its own, with no connection to anything, is a **standalone machine**. It can only use its own files and programs. The moment you connect it to another device, it becomes **networked**, and it can share data.

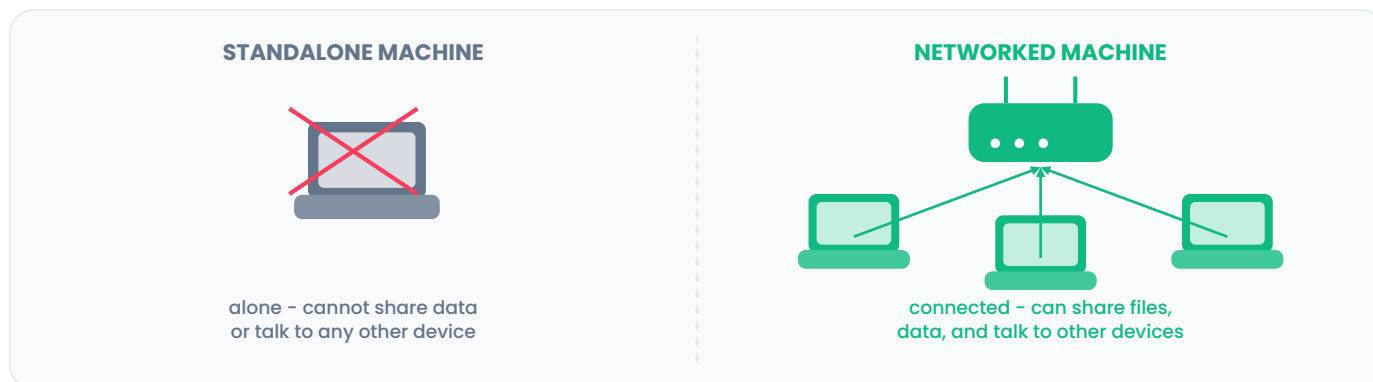


Fig 0.1. Alone, a machine can only talk to itself.

What "Internet" literally means

Internet = **Inter**connected **Net**work. It is simply a **network of networks**: millions of smaller networks (homes, offices, universities, data centers) all joined together so that any connected device can reach any other connected device.

THINK OF IT LIKE

A huge web of roads connecting every city and village on Earth. Your home Wi-Fi is your street. It joins a city road (your local ISP), which joins a national highway (your country's ISP), which joins an international highway system that circles the globe.

The simple picture: you send, you receive

At the most basic level, using the Internet is just this: your device **sends** data out, and **receives** data back, through a chain of connected networks.

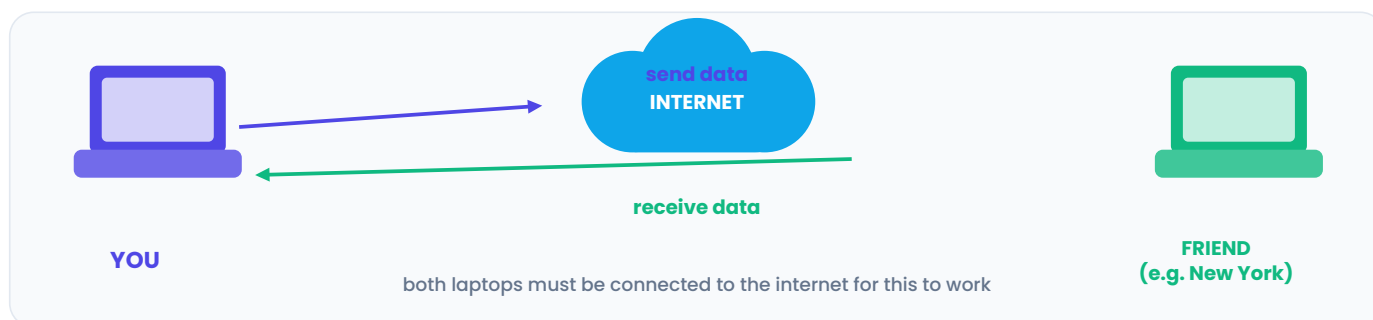


Fig 0.2. You and a friend, connected through "the internet" in the middle.

How continents actually connect: cables and satellites

The Internet is not magic floating in the sky. It is real, physical infrastructure. Two things connect the continents:

- **Submarine cables:** fiber-optic cables laid along the ocean floor, connecting country to country.
- **Satellites:** signals bounced off satellites in space (for example Starlink).

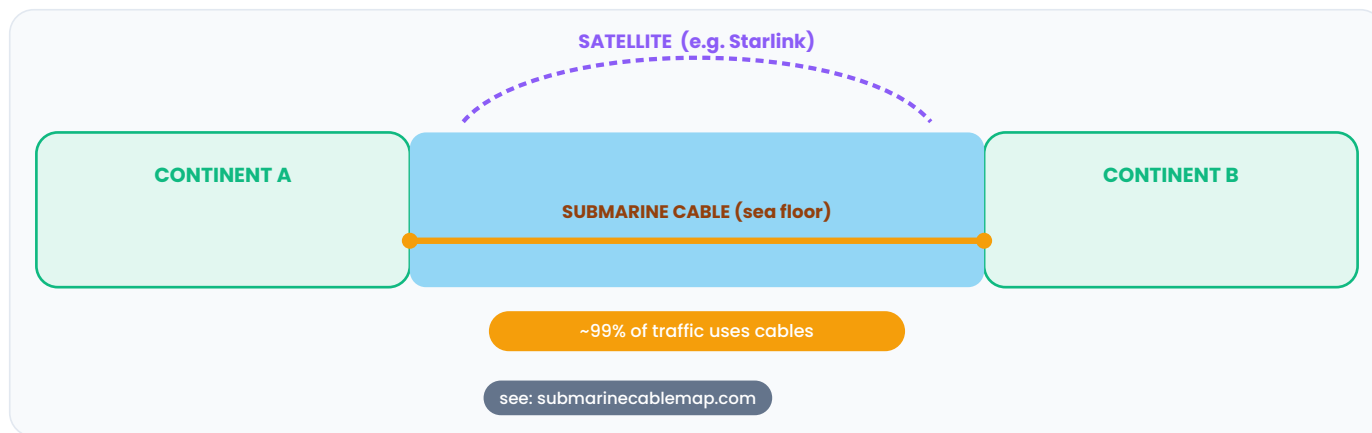


Fig 0.3. Two ways to cross an ocean.

KEY IDEA

Submarine cables carry the huge majority of international internet traffic (close to 99%), because fiber is far faster and cheaper per byte than satellite links. Satellites matter most for remote places cables cannot reach (ships, islands, rural areas). You can see the real map of these cables at submarinecablemap.com.

Who owns all this cable? The ISP tiers

Nobody owns "the Internet" as a whole. It is built and maintained by thousands of companies, organized loosely into three tiers.

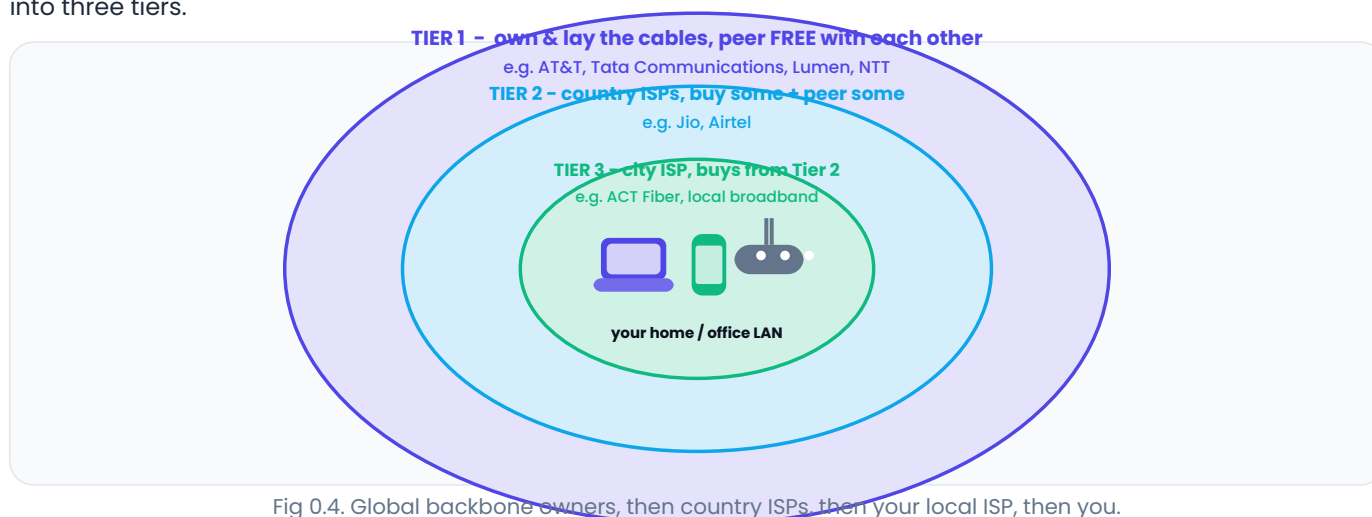


Fig 0.4. Global backbone owners, then country ISPs, then your local ISP, then you.

Tier	Who they are	What they do	Example
Tier 1	Global backbone owners	Own and lay the big international cables. They connect to each other for free (peering), because they all benefit equally.	AT&T, Tata Communications, Lumen, NTT, Telia
Tier 2	Country / regional ISPs	Buy access (transit) from Tier 1 networks, and sometimes peer with other Tier 2 networks directly.	Jio, Airtel
Tier 3	City / local ISPs	Buy access from Tier 2 and sell internet connections directly to homes and offices.	ACT Fiber, local cable broadband

Peering vs transit, and the IXP

Networks connect to each other in two ways: **peering** (we both benefit, so it is free) and **transit** (you pay to use someone else's network to reach the rest of the Internet). This often happens physically inside an **IXP (Internet Exchange Point)**, a building where many networks plug into the same switches to exchange traffic directly.

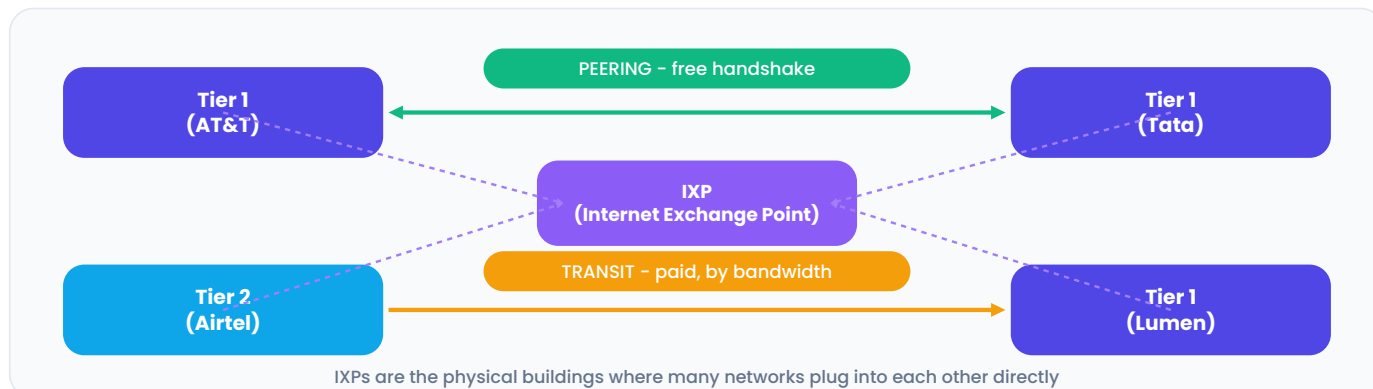


Fig 0.5. Peering is a free handshake. Transit is a paid ride.

Why does "free" home Wi-Fi turn into a cost?

Inside your own home network (LAN), sharing files, photos and streams between your own devices is effectively free; it never leaves your house. The moment your request **leaves your home** and goes onto the Internet, it starts hopping across networks that someone has to build, run and maintain, and **that costs money**. That cost is why you pay your ISP every month, and why your ISP in turn pays for transit up the chain.

EXAMPLE

You can run your own local network for free: share files, photos, movies, even stream a video between your own laptop and TV at home, with no internet bill at all. But the instant you want to reach a friend in another city, your request has to cross networks you do not own, and each hop has a maintenance cost someone is paying for.

The full journey of a request

Putting it all together, here is what really happens when you open a website hosted in another country:

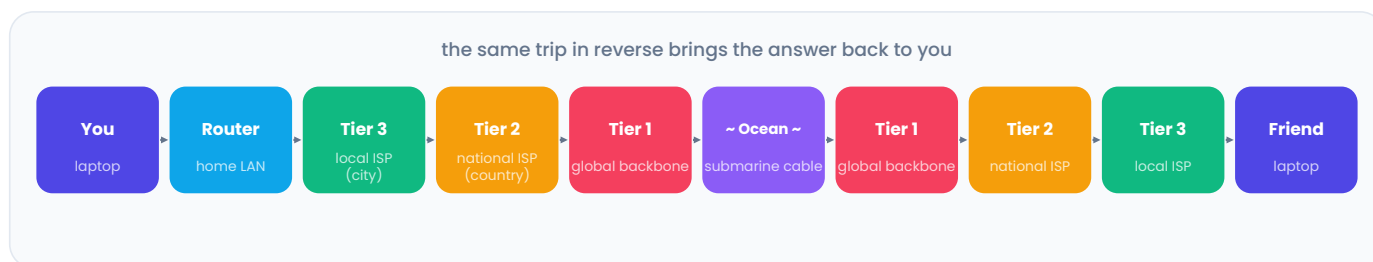


Fig 0.6. From your laptop, through three ISP tiers and an ocean, and back.

WHY DEVOPS CARES

When a website "feels slow," the delay could be at any one of these hops: your own Wi-Fi, your local ISP, a congested international cable, or the server itself. The troubleshooting tools in Section 15 (ping, traceroute) exist exactly to find out which hop is the problem.

02

Data Centers & The Cloud

What the cloud really is, and how AWS works

So where do the websites and apps you reach actually live? Inside **data centers**, run by companies like AWS, Azure and Google Cloud. This section explains what that really means, from the physical building up to "pro-level" cloud skills.

What is a data center?

A **data center** is a purpose-built building full of computers (**servers**), designed to keep them running non-stop: constant power (with backup generators), constant cooling, strong physical security, and multiple redundant internet connections so one failure never takes everything down.

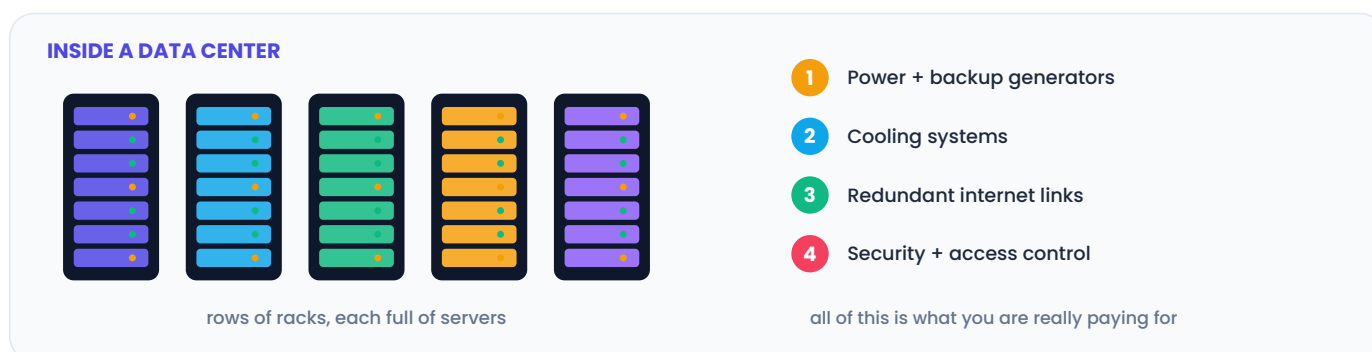


Fig 0.7. Rows of server racks, kept powered, cool and connected around the clock.

What does "the cloud" actually mean?

There is no literal cloud in the sky. **"The cloud" is just someone else's computer**, sitting in a data center far away, that you rent access to over the Internet.

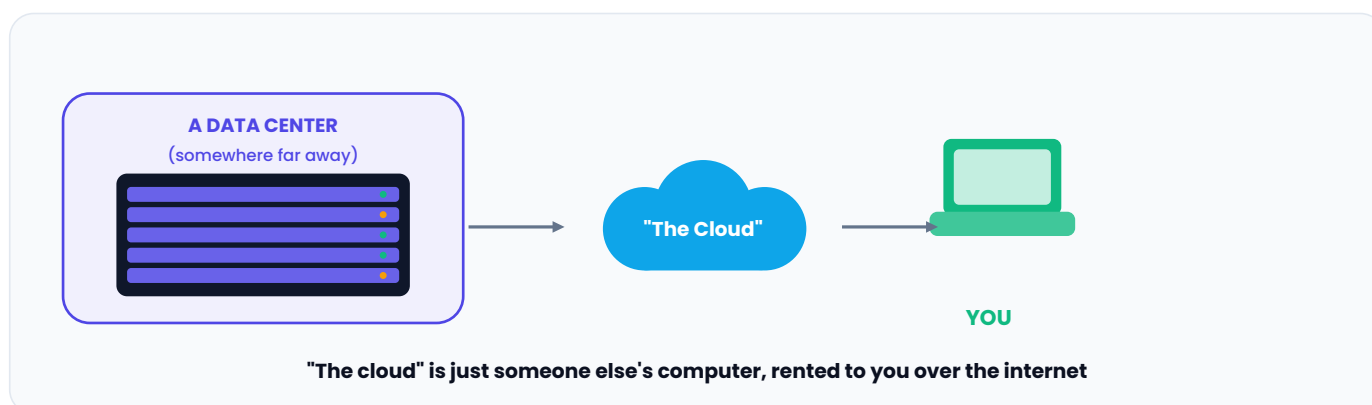


Fig 0.8. "Cloud computing" is renting a machine you never see or touch.

Why rent instead of own?

Buying, housing, powering, cooling and maintaining your own servers is expensive and slow to change. Cloud providers already built all of that, so you can **rent exactly the compute you need, pay only for what you use, and scale up or down in minutes.**

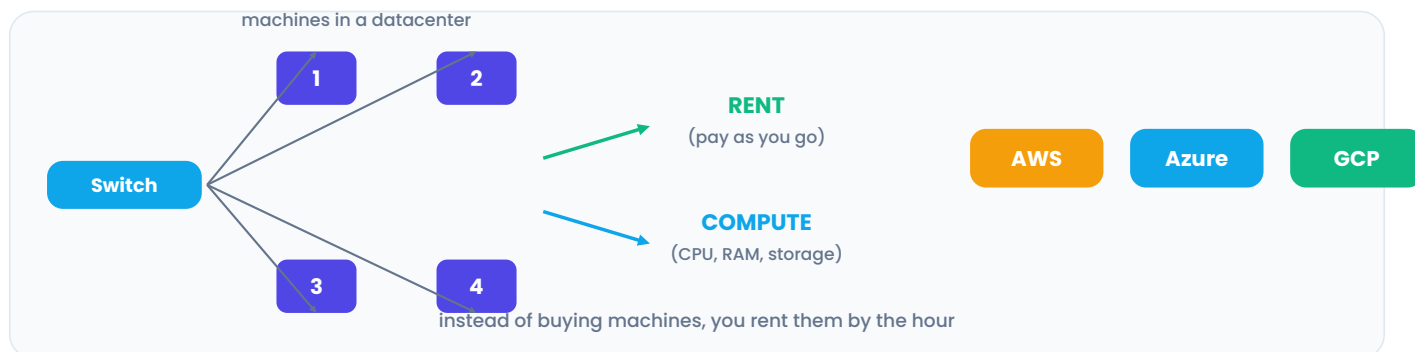


Fig 0.9. A cloud provider's switch and machines, rented to you on demand.

KEY IDEA

This is the same idea as the tier system in Section 1: you do not have to build and own the whole Internet backbone to use it, and you do not have to build and own a data center to use one. You rent what you need, from people who built it at scale.

The three service models: IaaS, PaaS, SaaS

Cloud services differ in **how much you manage yourself** versus how much the provider manages for you.

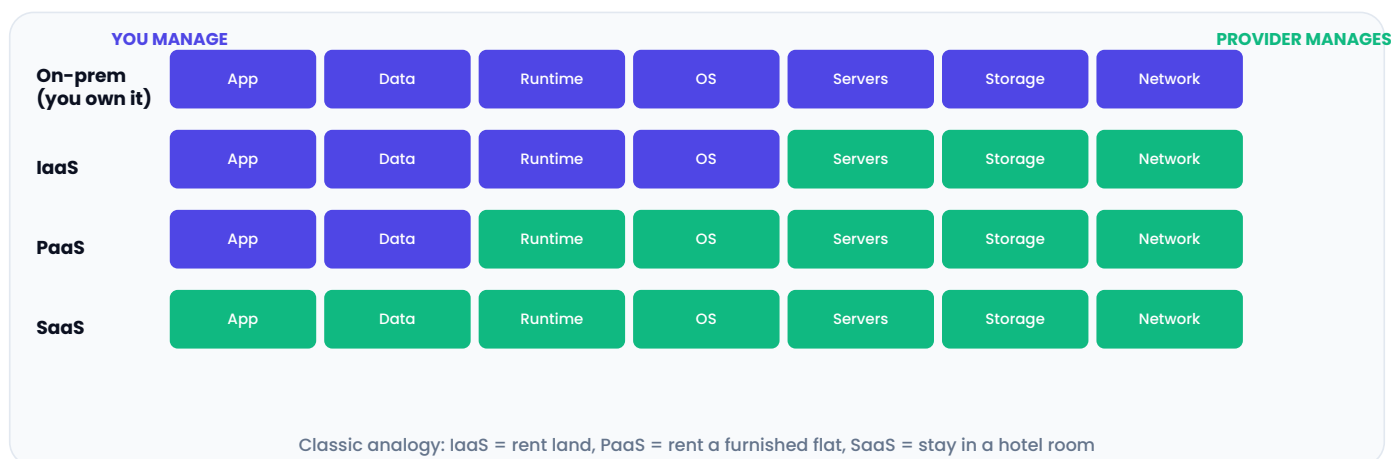


Fig 0.10. The more "as a Service" you buy, the less you manage yourself.

Model	You get	You still manage	Example
IaaS (Infrastructure)	Raw virtual servers, storage, network	OS, runtime, your app	AWS EC2, Azure VMs
PaaS (Platform)	A ready platform to run code on	Just your app and data	AWS Elastic Beanstalk, Heroku
SaaS (Software)	A finished, ready-to-use app	Nothing, just use it	Gmail, Slack, Netflix

EXAMPLE

Think of pizza. **IaaS** is buying ingredients and using your own kitchen. **PaaS** is ordering a pizza kit where the dough and sauce are ready and you just bake it. **SaaS** is ordering a pizza and it simply arrives at your door.

Public, private and hybrid cloud

Kind	Meaning
Public cloud	Shared infrastructure, rented from a provider like AWS, Azure or GCP
Private cloud	Infrastructure dedicated to one organization, often on their own premises
Hybrid cloud	A mix: some systems in a private data center, some in the public cloud, connected together

How AWS is organized: regions and availability zones

- A **Region** is a geographic area with its own group of data centers (for example **ap-south-1** = Mumbai).
- Each Region has 2 or more **Availability Zones (AZs)**: physically separate data centers inside that Region, so one fire or power cut does not take down the whole Region.
- You choose a Region close to your users for lower latency, and spread your servers across AZs for reliability.

The building blocks of AWS (and most clouds)

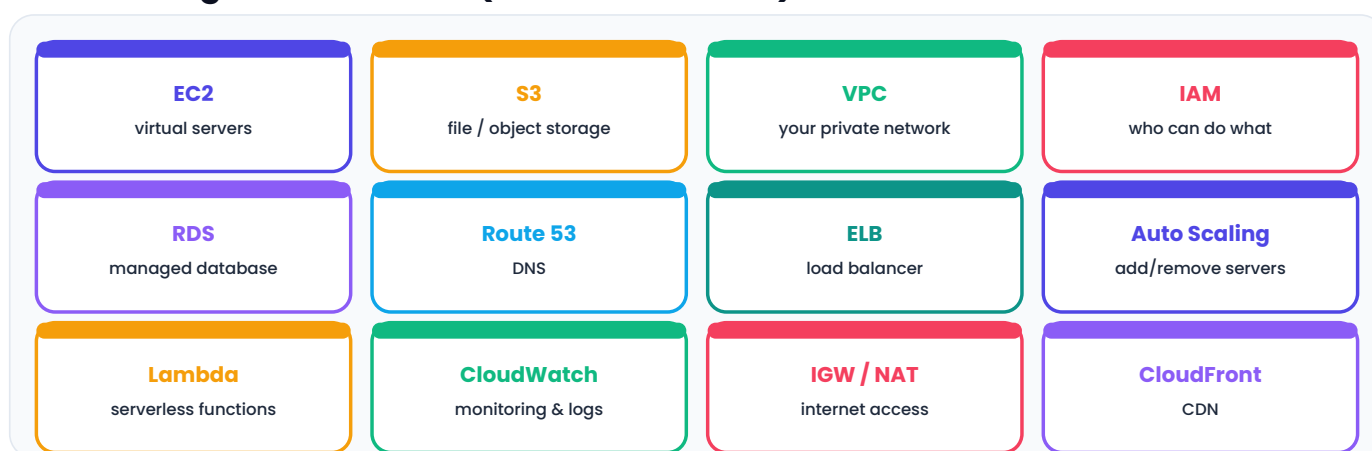


Fig 0.11. The AWS services you will meet again and again.

How a simple web app is built from these pieces

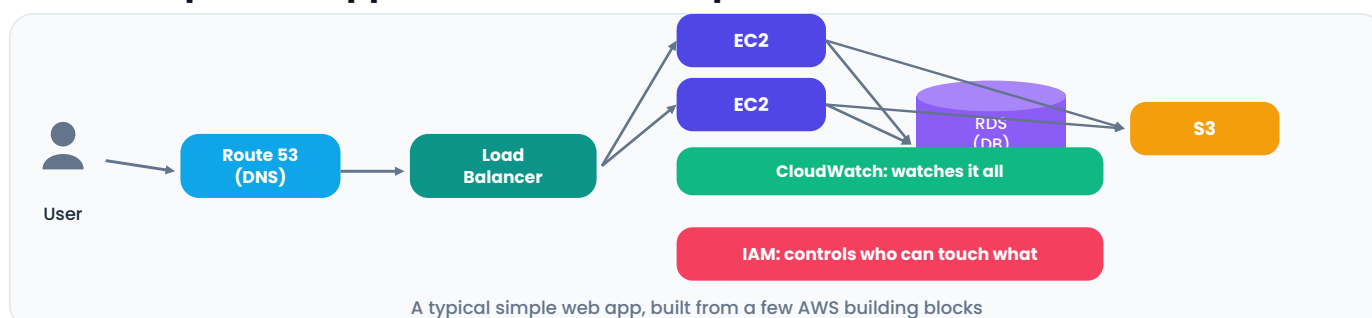


Fig 0.12. Route 53 finds it, the load balancer shares it, EC2 runs it, RDS stores it, S3 holds files, IAM guards it, CloudWatch watches it.

From scratch to pro: a learning order

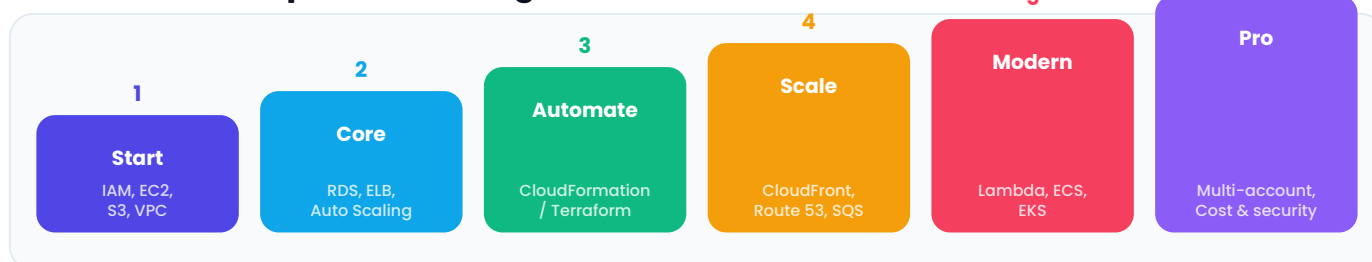


Fig 0.13. A sensible order to learn AWS in.

Stage	Learn	Why
1. Start	IAM, EC2, S3, VPC	The four services almost everything else is built on: identity, compute, storage, network
2. Core	RDS, Elastic Load Balancer, Auto Scaling	A real, reliable, scalable web app
3. Automate	CloudFormation or Terraform	Build your infrastructure from code, not clicks (see the System Design notes)
4. Scale	CloudFront (CDN), Route 53 routing, SQS (queues)	Serve a global audience fast and handle spikes
5. Modern	Lambda (serverless), ECS / EKS (containers)	Run code without managing servers at all
6. Pro	Multi-account setup, cost optimization, security (GuardDuty, Config)	Run AWS safely and cheaply at a company scale

WHY DEVOPS CARES

Whichever cloud you use, the core mental model is always the same: **compute** (servers) + **storage** (files/databases) + **network** (VPC, DNS, load balancers) + **identity** (who can do what). Learn those four well in any one cloud, and the others become easy to pick up.

03

Networking Basics

What a network is and the types of networks



A **network** is two or more devices (computers, phones, printers, servers) connected together so they can **talk to each other and share things**. The connection can be a cable or wireless (Wi-Fi).

THINK OF IT LIKE

A group chat. Everyone in the group is connected, anyone can send a message, and everyone can share photos and files. A network is a group chat for computers.

What is a network used for?

- **Communicate:** let connected devices send data to each other.
- **Share devices:** many users can use one printer, scanner, or a music / video server.
- **Share files and services:** websites, databases and apps live on servers that others reach over the network.

The Internet is the largest network in the world. It is called "**the network of networks**" because it joins millions of smaller networks together.

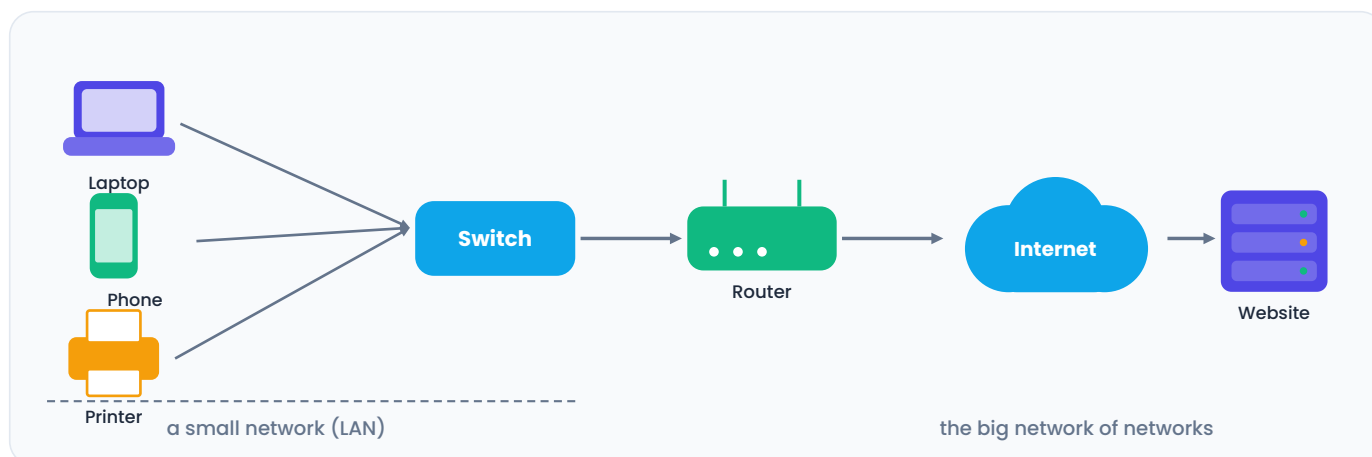


Fig 1. A small home network joins the Internet, which reaches websites everywhere.

Client and server

Almost everything on a network is a conversation between a **client** (asks for something) and a **server** (answers).

EXAMPLE

In a restaurant you are the **client**: you ask for food. The kitchen is the **server**: it prepares and serves it. Your browser (client) asks a website (server) for a page.

Types of networks

Networks are named by the area they cover. The main two are **LAN** and **WAN**.

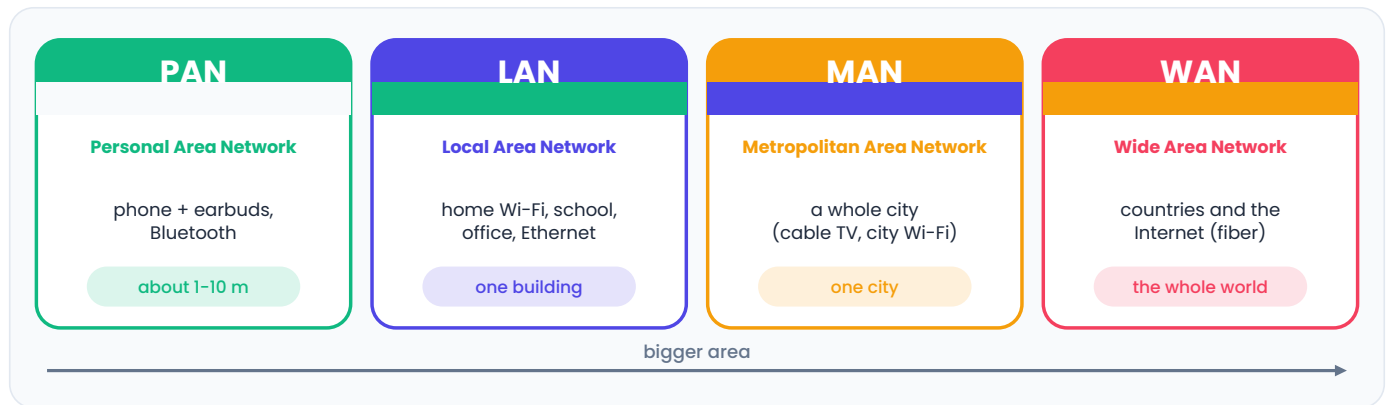


Fig 2. Networks by size.

Type	Full name	What it covers	Example
PAN	Personal Area Network	Around one person	Phone + Bluetooth earbuds
LAN	Local Area Network	Connects computers in a limited area like a home, school or office	Wi-Fi, Ethernet
MAN	Metropolitan Area Network	Used in a metropolitan area (a city)	City-wide cable network
WAN	Wide Area Network	Extends networks over a large geographic area	Optical fiber cable, the Internet
SONET	Synchronous Optical Network	A standard for sending data over optical fiber, used by telecom companies for long-distance links	Backbone fiber links

Network layouts (topologies)

A **topology** is the shape of how devices are connected. Today almost every office and home uses the **star** layout: every device connects to one central switch or router.

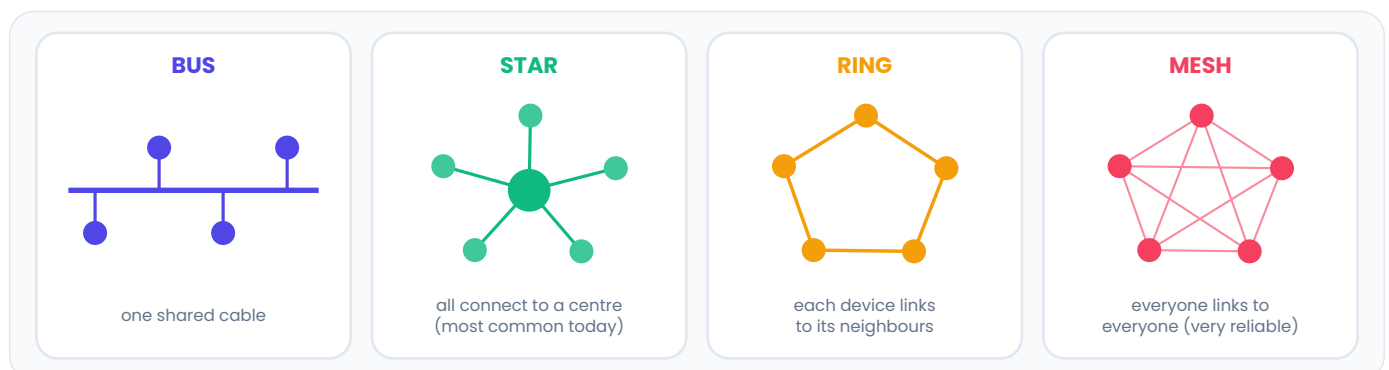


Fig 3. The four classic layouts.

Speed words you will hear

Three words describe how good a network feels. Learn them now; you will use them every day in DevOps.

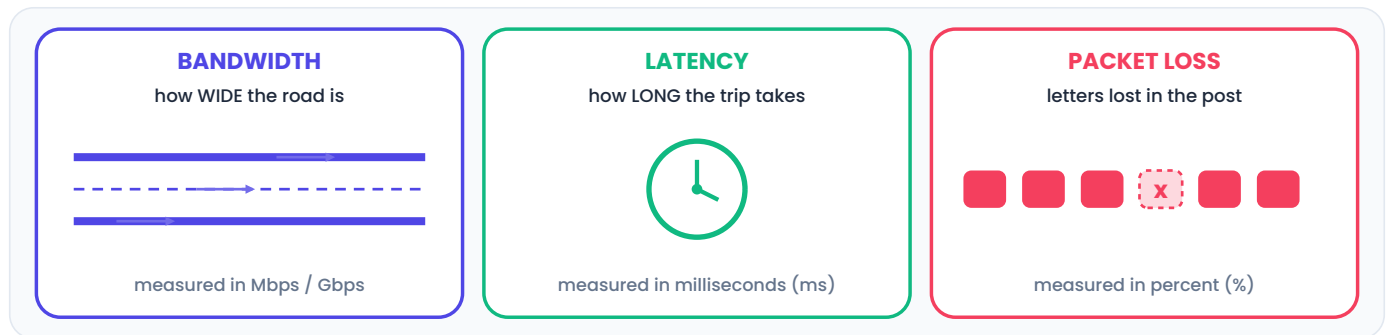


Fig 4. Bandwidth, latency and packet loss.

Word	Plain meaning	Good / bad
Bandwidth	The maximum amount of data per second the link can carry	Higher is better
Latency	The time one piece of data takes to travel and arrive (ping time)	Lower is better
Throughput	How much data actually gets through per second right now	Close to bandwidth is good
Packet loss	The share of data pieces that never arrive	0% is ideal

WHY DEVOPS CARES

A website feels slow for different reasons. **High latency** = far away server. **Low bandwidth** = big file on a thin link. **Packet loss** = bad Wi-Fi or an overloaded link. Knowing which one it is tells you what to fix.

04

Network Devices

Switch, router, modem, hub and friends

Networks are built from a few simple boxes. Here is how a normal home network is wired.

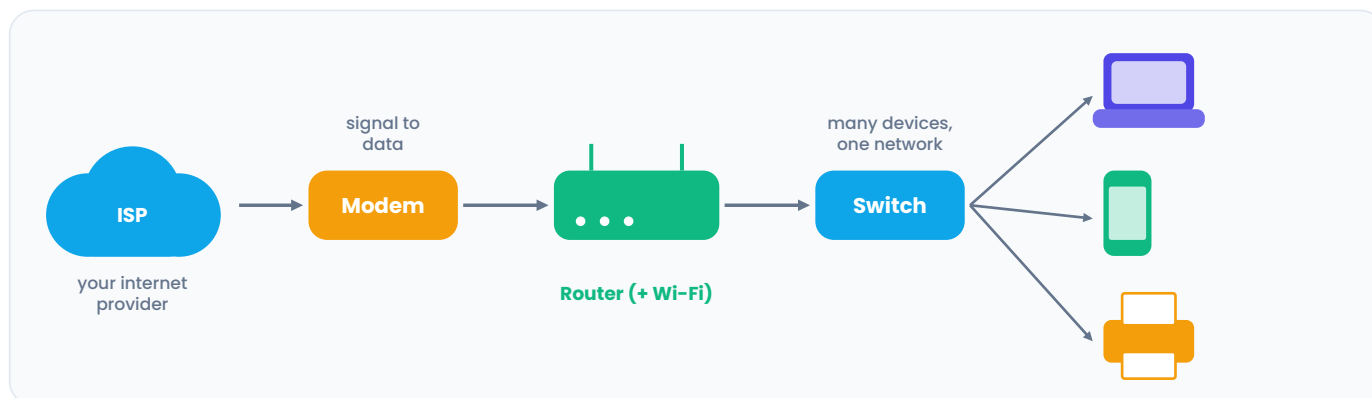


Fig 5. From your internet provider (ISP) to your devices.

The devices, one by one

Device	What it does (simple)	Think of it like
Switch	Connects two or more computers in the same network . It learns each device's MAC address and sends data only to the right one.	A hotel receptionist who knows every room number
Router	Connects one network to another (for example your home network to the Internet) and picks the path for data.	A road junction with signposts
Modem	Does modulation and demodulation : turns the signal on your ISP's line into digital data and back.	A translator between two languages
Hub	A dummy device. It just broadcasts every signal to all connected computers. Old and rarely used.	A person shouting in a room
NIC	Network Interface Card. Connects your computer to the network. Today it is built into the motherboard (or is a Wi-Fi card). It has a MAC (Media Access Control) address .	The network plug / antenna of your computer
Bridge	Connects multiple LANs into one larger LAN, reduces broadcasting and stores MAC addresses. Now obsolete : replaced by the switch.	An old footbridge, replaced by a highway

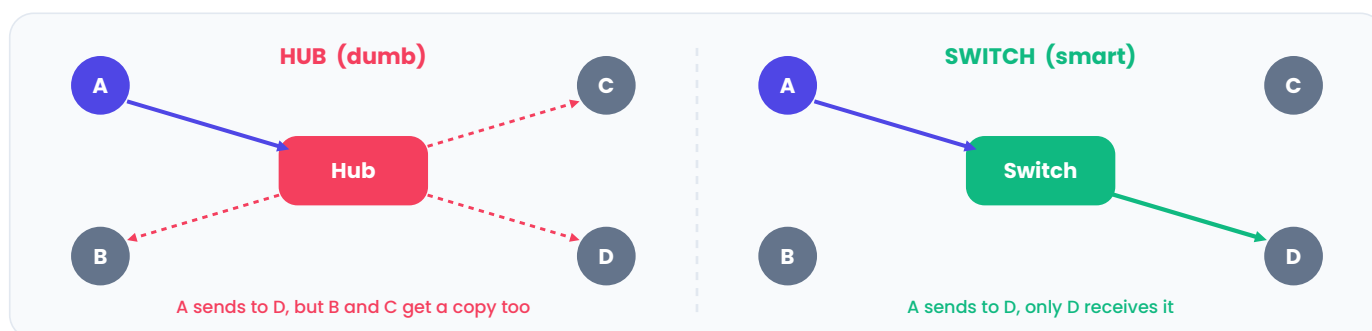


Fig 6. Hub shouts to everyone. Switch delivers to one device.

PRO TIP

Your home "Wi-Fi box" is usually **four devices in one**: modem, router, switch and a wireless access point. In a company they are separate boxes, and you will meet more: **firewalls**, **load balancers** and **VPN gateways** (Section 13).

05

Protocols, Packets & MAC

The rules and the envelopes of the internet



What is a protocol?

A **network protocol** is a set of rules that decides **how data is sent between devices**. Both sides must follow the same rules, just like two people must speak the same language.

Examples: **HTTP** (web pages), **TCP** (reliable delivery), **IP** (addressing), **FTP** (file transfer), **SMTP** (sending email).

THINK OF IT LIKE

Traffic rules. Every driver follows the same rules (stop at red, drive on one side), so strangers can share the road safely. Protocols are the traffic rules of the network.

Data travels in small pieces

Computers do not send one huge file in one go. They cut it into small pieces called **packets**, label each one with an address, and send them. The receiver puts them back together.

EXAMPLE

You want to post a 600-page book. You cannot fit it in one envelope, so you send it as 30 envelopes. Each envelope has the same address and a number (page 1, page 2, ...). The friend opens them and rebuilds the book. That is exactly what happens to a photo you send.

Courier to next stop

MAC: next device

Street address

IP: which computer

Envelope + door no.

port: which app

Letter

your message (data)

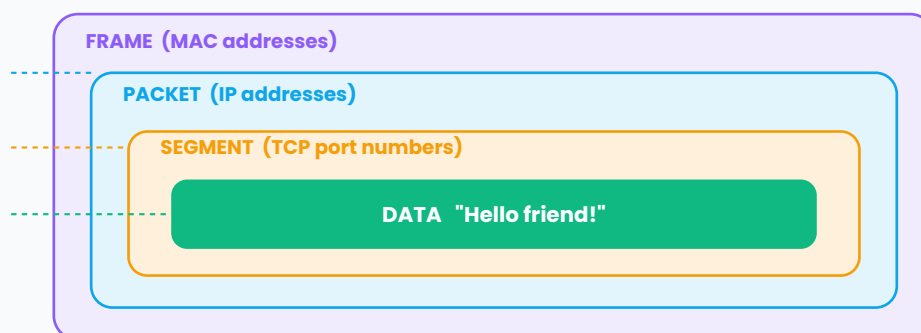


Fig 7. Data is wrapped in layers of "envelopes". Each layer adds its own address.

Two kinds of address: MAC and IP

MAC ADDRESS

00:1A:2B:3C:4D:5E

the serial number of your network card

- 1 burned into the hardware
- 2 works inside one local network
- 3 like your name

IP ADDRESS

192.168.1.10

the home address of your device on a network

- 1 given by the network, can change
- 2 works across networks
- 3 like your street address

Fig 8. MAC = who the hardware is. IP = where it is on the network.

	MAC address	IP address
What	Hardware (physical) address of the network card	Logical address given by the network
Changes?	No, fixed in the card (MAC = Media Access Control)	Yes, can change when you join another network
Used for	Delivery inside one local network	Delivery across networks and the Internet

	MAC address	IP address
Looks like	00:1A:2B:3C:4D:5E	192.168.1.10

06

IP Addressing

Every device needs an address



What is an IP address?

An **IP (Internet Protocol) address** is a unique number given to each device on a network so devices can find and talk to each other. It is like the **home address** of the device on the internet or on your local network.

IPv4: the classic address

- A **32-bit** number written as **four numbers separated by dots**, for example 123.89.46.7.
- Each number is called an **octet** (8 bits) and goes from **0 to 255**.
- It is easy to remember, but there are only about **4.3 billion** of them.

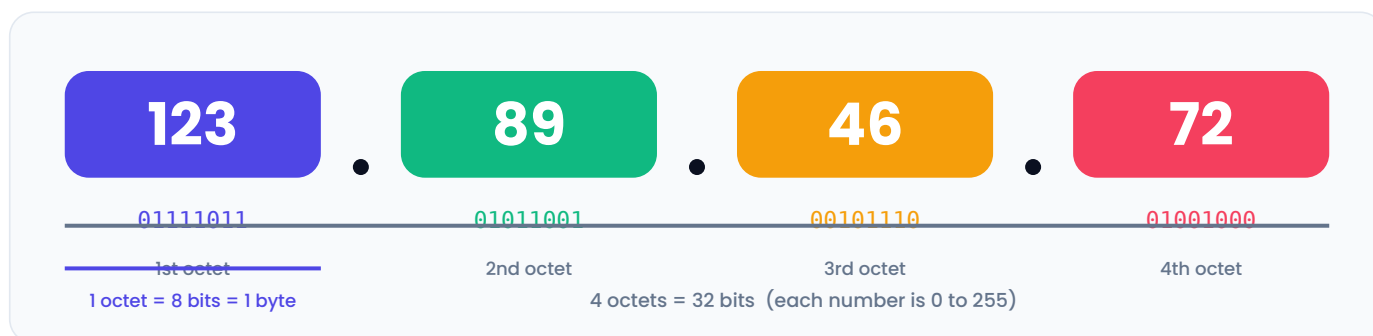


Fig 9. An IPv4 address: 4 octets, 32 bits.

How a computer reads an octet (binary)

Computers only know 0 and 1. Each of the 8 positions in an octet has a value: 128, 64, 32, 16, 8, 4, 2, 1. Add up the positions that have a 1.

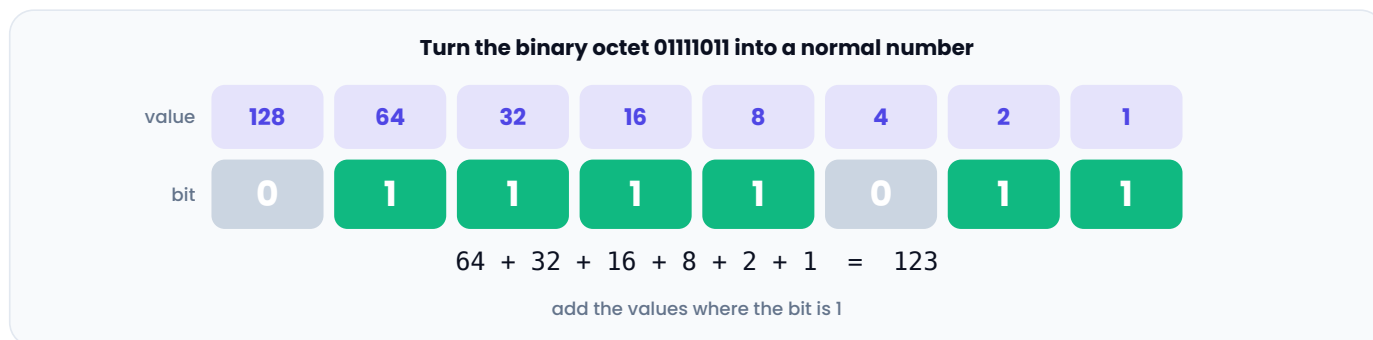


Fig 10. Binary to decimal: 01111011 = 123.

IPv6: the new, bigger address

IPv4 is running out, so **IPv6** was made to replace it. It is **128 bits**, written as **eight groups of hexadecimal** numbers. It gives a vastly larger pool of addresses.

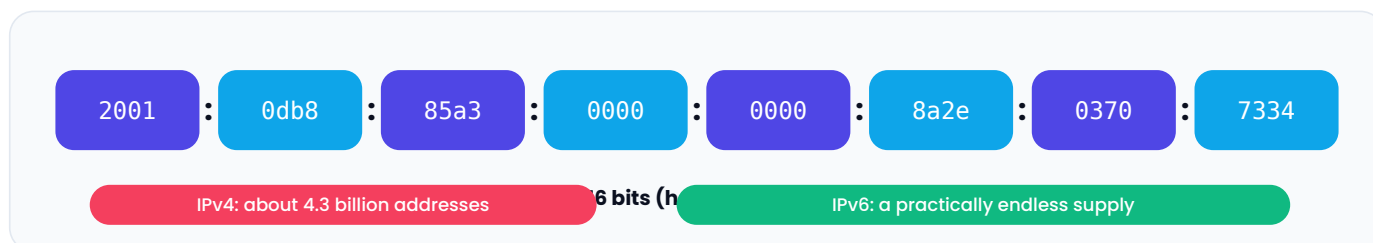


Fig 11. An IPv6 address.

Public IP and private IP

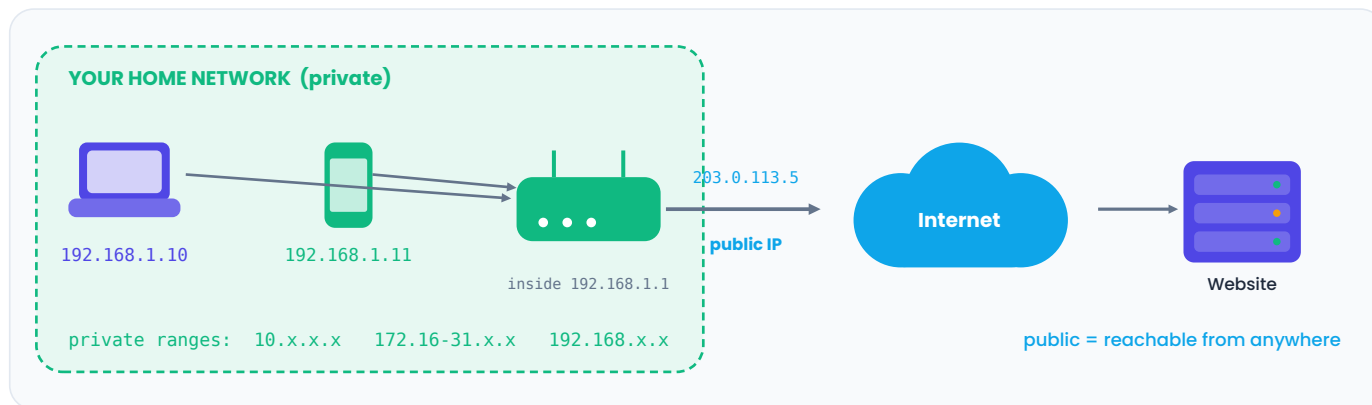


Fig 12. Your devices have private IPs. The router shows one public IP to the world.

	Public IP	Private IP
Used	To identify devices on the internet	Inside private networks (home, office, cloud)
Who gives it	Your ISP (internet provider)	Your router or network admin
Reachable from internet?	Yes, accessible globally	No
Ranges	Everything else	10.x.x.x, 172.16.x.x - 172.31.x.x, 192.168.x.x

Static IP and dynamic IP

	Static IP	Dynamic IP
How it is set	Set manually , never changes	Given automatically by a DHCP server
Changes?	No	Yes, changes periodically
Used for	Servers and devices that need a consistent address	Home devices: phones, laptops

Special addresses worth knowing

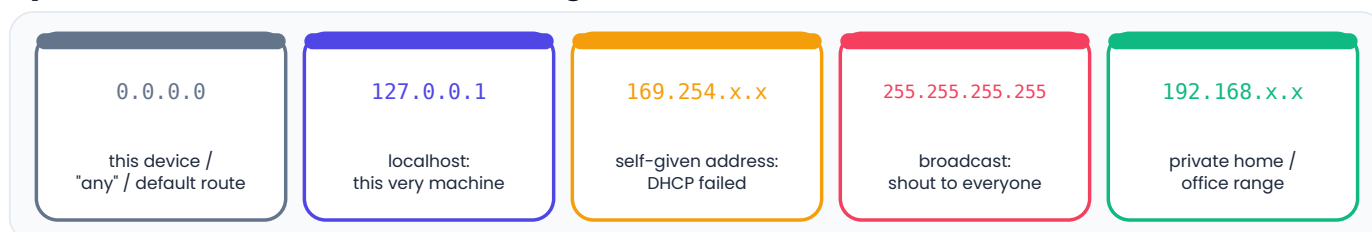


Fig 13. A few addresses with a special meaning.

Loopback: talking to yourself

Loopback lets a device talk to itself. The address **127.0.0.1** is called "**localhost**". Any address in **127.x.x.x** loops back to the same device. Developers use it to test apps without needing a real network.

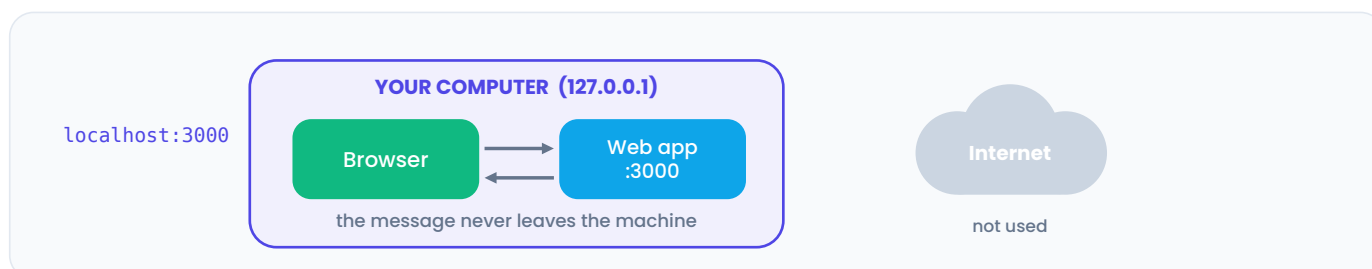


Fig 14. localhost never leaves your machine.

Look at your own IP address

System	Command
Linux	<code>ip a or hostname -I</code>
Windows	<code>ipconfig</code>
macOS	<code>ifconfig</code>

Sample output on Windows:

```
IPv4 Address. . . . . : 192.168.43.195
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.43.1
```

Your **default gateway** is the router: the door your traffic uses to leave the local network.

IP address classes (IPv4 only)

An organization called **IANA** (Internet Assigned Numbers Authority) divided IPv4 addresses into **five classes**, based on the **first number**. The class tells you how big the network is meant to be.

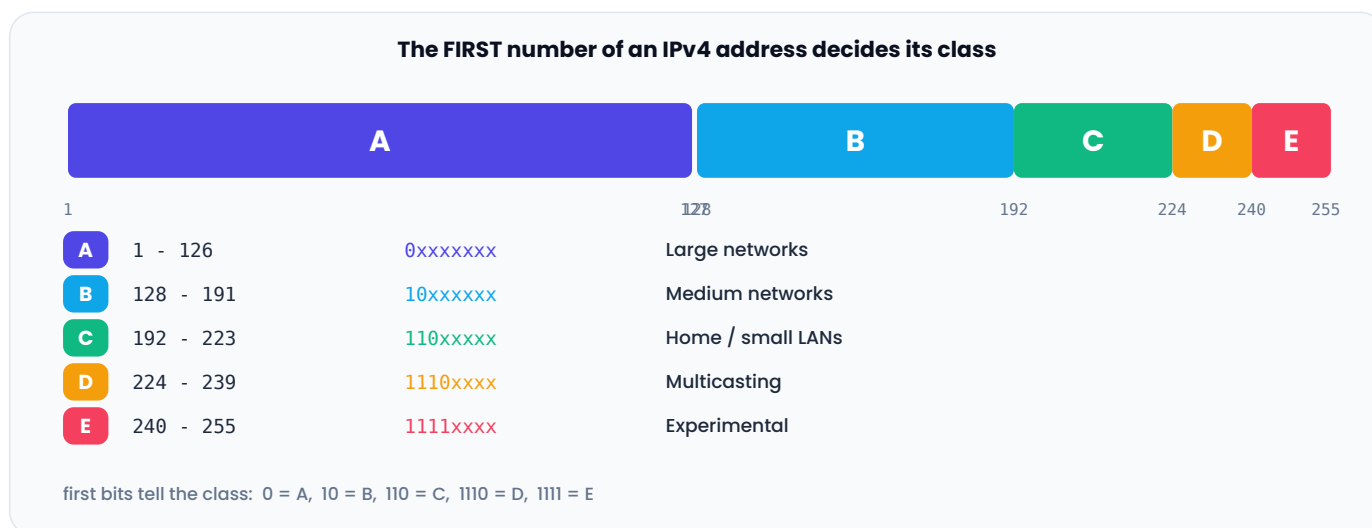


Fig 15. The five classes on a 0-255 line.

Class	Range	Purpose
A	1.0.0.0 - 126.0.0.0	Large networks, like big organizations
B	128.0.0.0 - 191.255.0.0	Medium-sized networks
C	192.0.0.0 - 223.255.255.0	Small networks, like home or business LANs
D	224.0.0.0 - 239.255.255.255	Reserved for multicasting
E	240.0.0.0 - 255.255.255.255	Experimental, used for research

Where do those ranges come from? (binary)

The first octet is 8 bits with values 128, 64, 32, 16, 8, 4, 2, 1. The class depends on how many of the **first bits are 1**:

First bits	Smallest	Largest	Result
0xxxxxxx	0 (all zeros)	127 (64+32+16+8+4+2+1)	Class A: 0 - 126 (0 and 127 are special)
10xxxxxx	128	191 (128+63)	Class B: 128 - 191
110xxxxx	192 (128+64)	223	Class C: 192 - 223
1110xxxx	224	239	Class D: 224 - 239
1111xxxx	240	255	Class E: 240 - 255

WATCH OUT

0.0.0.0 is not part of the usable Class A range; it has a special meaning ("this device" / "any address"). **127.0.0.0 - 127.255.255.255** is reserved for **loopback**.

Network ID and Host ID

Every IP address has two parts: the **Network ID** (which network) and the **Host ID** (which device inside it). Any device that gets an IP is called a **host**. How many octets belong to the Network ID depends on the class.

NETWORK ID = the street HOST ID = the house number

Class A



Class B



Class C



EXAMPLE

The address works like "**Street name, house number**". The Network ID is the street, the Host ID is the house. Two houses on the same street can visit each other directly. To reach another street you need a road junction: a **router**.

Same network or not?

Device A	Device B	Network ID of A / B	Result
17.0.0.1	17.0.4.2	17 / 17 (Class A, same)	Connect directly , no router needed
17.0.0.1	192.168.1.5	17 / 192.168 (different)	A router is required

KEY IDEA

Direct connection: same Network ID = no router. **Router requirement:** different Network IDs = need a router. Classes are the old way of splitting; modern networks use **CIDR** (next section), but classes help you understand the idea.

07

Subnetting & CIDR

Cutting big networks into small ones



What is subnetting?

Subnetting divides one big network into smaller, easier-to-manage networks called **subnets**.

THINK OF IT LIKE

A big office floor. Instead of one huge open hall where everyone hears everything, you build rooms: Accounts, Engineering, Guests. Each room is quieter, safer, and easier to manage. Subnets are the rooms.

Why do it?

- **Better performance:** less broadcast traffic, because broadcasts stay inside a subnet.
- **More security:** separate departments or functions (for example web servers and databases).
- **Efficient IP use:** give each subnet only the addresses it needs.

The subnet mask and the "/" number

The **subnet mask** tells which part of an address is the **network** and which part is the **host**. The short form **/24** (a "prefix") simply counts how many bits belong to the network.

IPv4 has 32 bits. Blue = network part, amber = host part

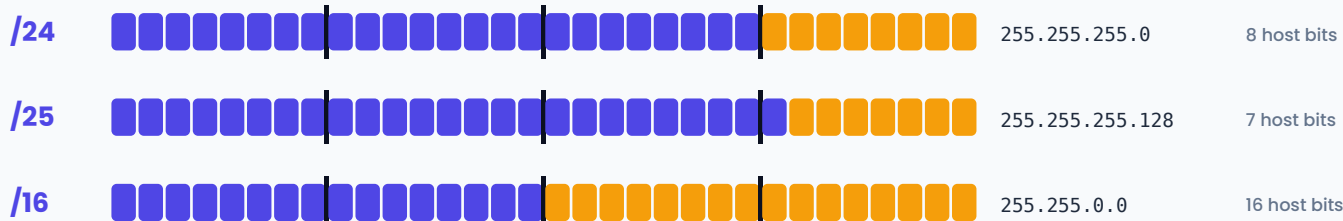


Fig 17. A /24 uses 24 bits for the network and 8 bits for hosts.

EXAMPLE

192.168.1.0/24 is a Class C network. **/24** means mask **255.255.255.0**, so $32 - 24 = 8$ bits for hosts, and $2^8 = 256$ addresses (192.168.1.0 to 192.168.1.255).

Example: split a /24 into two /25 subnets

To make **two equal subnets**, use **/25**. That leaves $32 - 25 = 7$ bits for hosts, so each subnet has $2^7 = 128$ addresses.

192.168.1.0/24 = 256 addresses (.0 to .255)



Fig 18. One /24 becomes two /25 subnets.

Subnet	Range	Subnet mask	Total IPs	Usable host IPs
192.168.1.0/25	192.168.1.0 - 192.168.1.127	255.255.255.128	128	126
192.168.1.128/25	192.168.1.128 - 192.168.1.255	255.255.255.128	128	126

Why 126, not 128? The **first** address is the **network address** (the name of the subnet) and the **last** is the **broadcast address** (shout to everyone). Neither can be given to a device.

Subnetting in 4 easy steps

1. **Host bits** = 32 - prefix. (/25 gives 7)
2. **Total addresses** = 2 to the power of host bits. ($2^7 = 128$)
3. **Usable hosts** = total - 2. ($128 - 2 = 126$)
4. **Next subnet starts** after the block ends. (0 to 127, then 128 to 255)

Prefix	Mask	Host bits	Total	Usable
/24	255.255.255.0	8	256	254
/25	255.255.255.128	7	128	126
/26	255.255.255.192	6	64	62
/27	255.255.255.224	5	32	30
/28	255.255.255.240	4	16	14
/29	255.255.255.248	3	8	6
/30	255.255.255.252	2	4	2

CIDR (Classless Inter-Domain Routing)

CIDR replaced the old class system (A, B, C). Instead of fixed sizes, you write any prefix you like, such as /22 or /27. It was introduced to **use IP addresses better** and **simplify routing**. Remember: **bigger prefix number = smaller network**.



Fig 19. Each step down makes the network smaller.

The most common prefixes

Prefix	Netmask	Number of addresses	Relation to class	Comment
/32	255.255.255.255	1	Class C / 256	A single host
/25	255.255.255.128	128	Class C / 2	
/24	255.255.255.0	256	Class C	
/23	255.255.254.0	512	Class C x 2	
/16	255.255.0.0	65,536	Class C x 256 = Class B	
/15	255.254.0.0	131,072	Class B x 2	
/8	255.0.0.0	16,777,216	Class B x 256 = Class A	
/0	0.0.0.0	4,294,967,296	Class A x 256	0.0.0.0/0 means the entire internet ; often used in firewall rules

WHY DEVOPS CARES

You choose CIDR blocks all the time: a cloud network (VPC) might be **10.0.0.0/16** with subnets like **10.0.1.0/24**. In a firewall rule, **0.0.0.0/0** means "**everyone on the internet**", so use it carefully (for example never for SSH).

08

The OSI Model

Seven layers, one message



A network model is a **map that splits the job of communication into layers**. Each layer does one small job and only talks to the layer above and below it.

The **OSI (Open Systems Interconnection) model** was developed by the **International Organization for Standardization (ISO)**. It has **7 layers**, each with specific functions and responsibilities.

THINK OF IT LIKE

Posting a letter. You write it (Application), put it in an envelope (Transport), add the address (Network), the post van takes it to the next stop (Data Link), and the road carries it (Physical). At the other end, everything happens in reverse.

LAYER	WHAT IT DOES	DATA UNIT	DEVICE
7 Application	talks to the user: web, mail	Message / data	Gateway
6 Presentation	format, encrypt, compress	Message / data	Gateway
5 Session	opens / keeps / closes sessions	Message / data	Gateway
4 Transport	reliable delivery, ports	Segment (UDP)	Gateway
3 Network	IP addresses, routing	Packet	Router
2 Data Link	MAC addresses, frames	Frame	Switch
1 Physical	raw bits on the wire	Bits	Cable

Remember (top to bottom): All People Seem To Need Data Processing

Fig 20. The 7 layers of the OSI model.

What each layer does

No.	Layer	Job in plain words	Data unit	Devices	Protocols
7	Application	Interfaces directly with the user and provides network services	Message / data	Gateway	HTTP, HTTPS, FTP, SMTP, DNS, SSH, DHCP
6	Presentation	Translates data formats; handles encryption and compression	Message / data	Gateway	SSL / TLS, MIME, JPEG / PNG
5	Session	Starts, maintains and ends conversations between applications	Message / data	Gateway	NetBIOS, RPC, PPTP
4	Transport	Reliable data transfer with error correction and flow control	Segment (TCP), datagram (UDP)	Gateway	TCP, UDP, SCTP
3	Network	Packet forwarding and routing using IP addressing	Packet	Router	IP, ICMP, ARP*, NAT, IGRP
2	Data Link	Delivery between directly connected nodes; error detection and flow control	Frame	Switch, bridge, NIC	Ethernet, Wi-Fi, PPP, HDLC, ARP
1	Physical	The physical connection: raw bits over cables, radio signals, etc.	Bits	Cables, modem, hub, NIC	Ethernet signaling, USB, Bluetooth

* ARP sits between layers 2 and 3. You will meet it in Section 11.

A message travels down, across, and up

Person **X** sends a message to Person **Y**. On the way down, every layer adds its own wrapper. On the way up, every layer removes it.

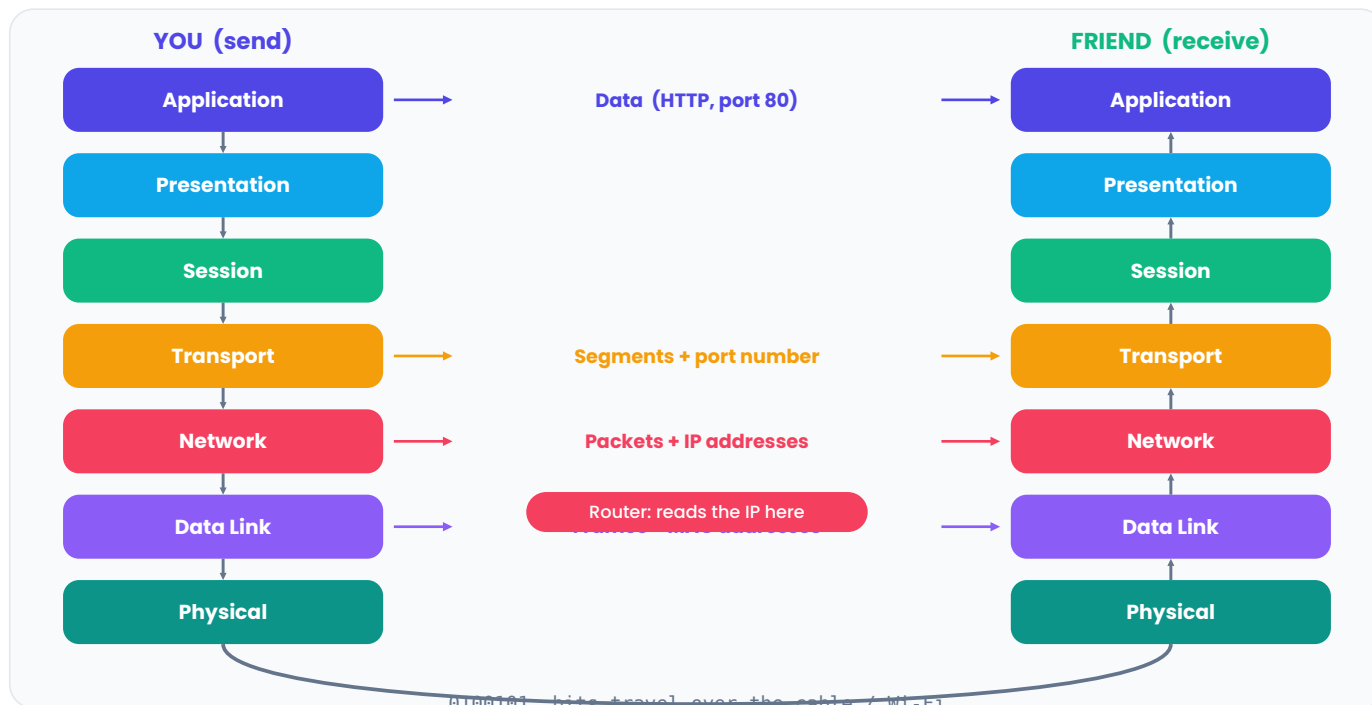


Fig 21. Down the stack on the sender, across the cable, up the stack on the receiver.

Step by step: from Person X to Person Y

Layer	Sender (X)	Receiver (Y)
7 Application	Prepares the message using the messaging app protocol	Shows the message in the app
6 Presentation	Encodes / encrypts it into a suitable format	Decodes / decrypts it into readable form
5 Session	Opens a session with Y's device	Verifies and keeps the session alive
4 Transport	Breaks it into segments, adds TCP / UDP header (port)	Reassembles the segments into data
3 Network	Adds source and destination IP addresses	Extracts IP addresses and checks routing info
2 Data Link	Makes frames, adds MAC addresses	Verifies and processes frames using MAC addresses
1 Physical	Converts frames into bits (0s and 1s) and sends them	Receives bits and rebuilds frames

EXAMPLE

In the handwritten-style flow of a web request: your browser sends HTTP on **port 80** (Application). Transport cuts it into **segments**. Network adds your **source IP** and the friend's **destination IP** as **packets**. A **router** connects the two networks. Data Link adds **MAC addresses** as **frames**. Physical sends **bits** over the cable. The friend's computer checks IP and MAC and passes it up.

Ports and layers

Port numbers are used mainly in the **Application and Transport layers**, to make sure data reaches the right service on a computer. Layers 2 to 6 manage connections and data formats and do not use port numbers.

Why DevOps engineers use the OSI model

When something breaks, check the layers **from the bottom up**. Each layer points to a different kind of problem.

Layer	Typical problem	How you check
1 Physical	Cable unplugged, Wi-Fi off	Link lights, <code>ip a</code>
2 Data Link	Wrong VLAN, bad switch port	Switch config, <code>arp -a</code>
3 Network	Wrong IP, no route, wrong subnet	<code>ping</code> , <code>traceroute</code> , <code>ip route</code>
4 Transport	Port closed, firewall blocking	<code>nc -zv host 443</code> , <code>ss -tuln</code>
7 Application	Website returns 500, bad DNS	<code>curl -I</code> , <code>dig</code> , app logs

09

The TCP/IP Model

The model the real internet uses

The **TCP/IP model**, also called the **Internet Protocol Suite**, is a simplified version of OSI with only **4 layers**. It is the model that **actually works in real life** and runs the Internet. OSI is great for learning; TCP/IP is what is really used.

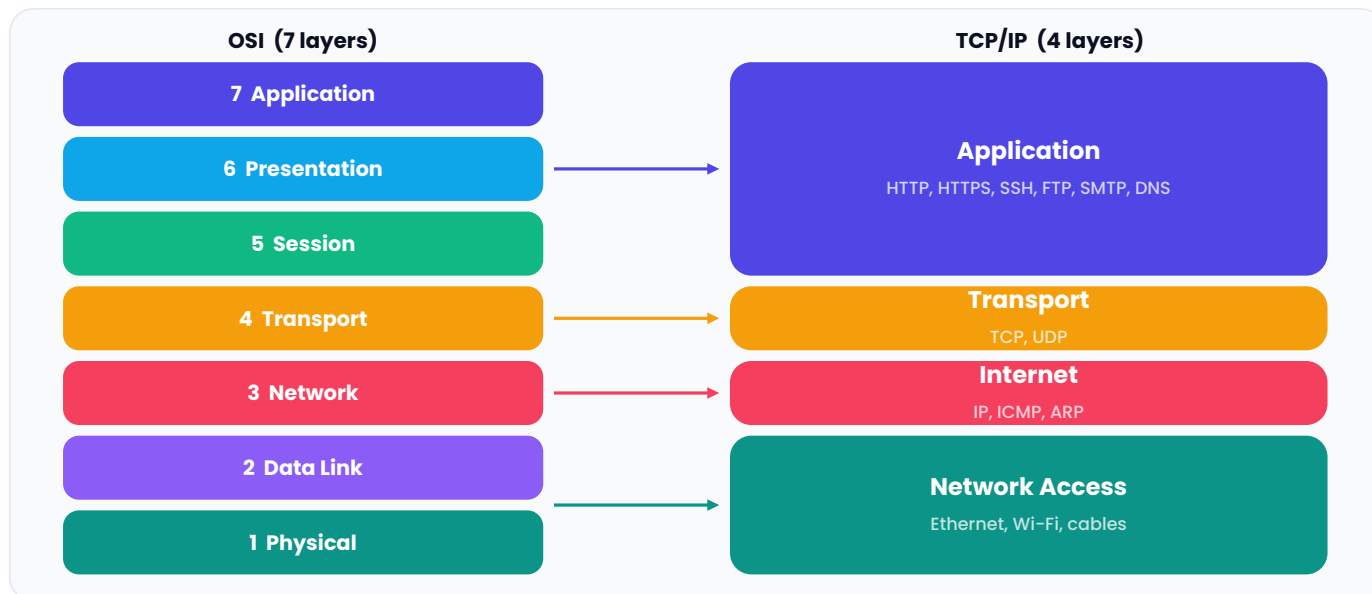


Fig 22. OSI (left) compared with TCP/IP (right).

TCP/IP layer	Equals this in OSI	What it does	Protocols
Application	Application + Presentation + Session	Generates the data and requests the connection	SMTP, HTTP / HTTPS, FTP, SSH
Transport	Transport	Sets up an error-free connection, splits data into smaller packets, gets acknowledgment of receipt	TCP, UDP
Internet	Network	Sends the packets, makes sure they are sent accurately, routes data to the correct network	IPv4 / IPv6, ICMP, ARP
Network Access	Data Link + Physical	Adds the destination MAC address, sends data between devices, handles the physical infrastructure	Ethernet, Wi-Fi

The journey of a message

The sender's data goes **down** the 4 layers (Layer 1 to 4 on the sending side), travels across the network, then goes **up** the 4 layers on the receiving computer.

	OSI model	TCP/IP model
Layers	7	4
Used for	Learning, teaching, troubleshooting vocabulary	The real working Internet
Name	Open Systems Interconnection	Internet Protocol Suite

KEY IDEA

When people say "Layer 3" or "Layer 7" (for example "a Layer 7 load balancer"), they use **OSI numbers**. When they build real systems, they follow **TCP/IP**. You need both in your head.

10

Ports & Protocols

HTTP, HTTPS, mail, FTP, SSH, TCP, UDP



Ports: which door to knock on

An IP address finds the **computer**. A **port number** finds the **program (service)** on that computer. Port numbers go from **0 to 65535**.

THINK OF IT LIKE

An apartment building. The IP address is the building address. The port is the door number of one flat. Mail for flat 443 (HTTPS) must not be delivered to flat 22 (SSH).

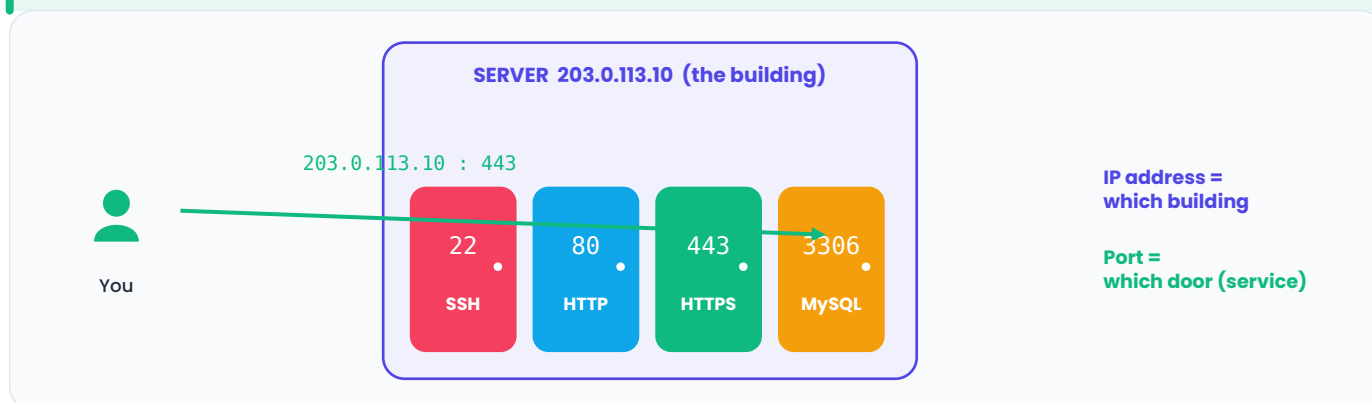


Fig 23. IP address + port = the exact service you want.

Range	Name	Used for
0 - 1023	Well-known ports	Standard services (80, 443, 22, 53 ...)
1024 - 49151	Registered ports	Applications (3306 MySQL, 5432 PostgreSQL ...)
49152 - 65535	Dynamic / private	Temporary ports your computer picks when it connects out

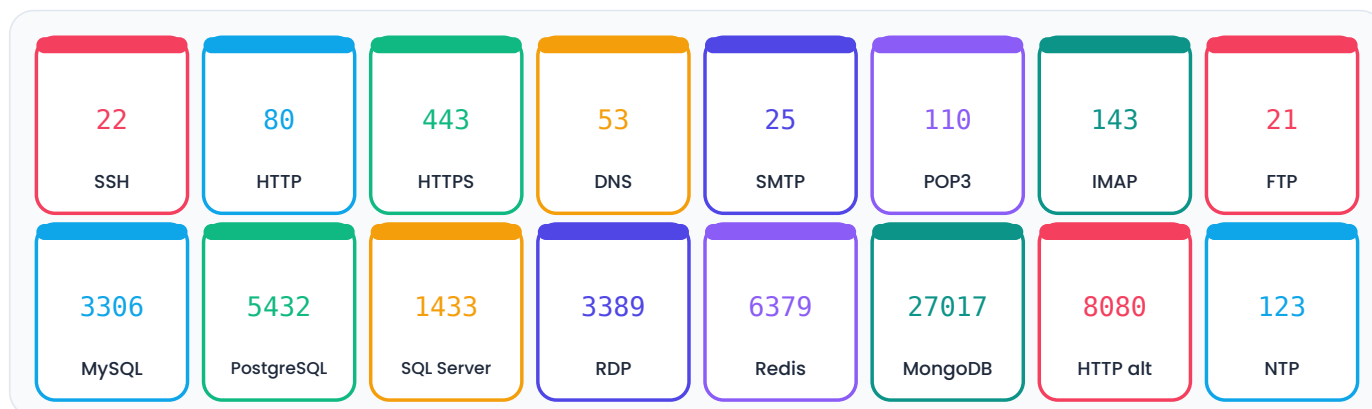


Fig 24. Common ports to remember.

HTTP: how the web talks

HTTP (Hypertext Transfer Protocol) is a **client-server** protocol. It tells the client how to ask the server for data and tells the server how to send it back. HTTP is **stateless**: the server does not keep any memory of the client between requests.

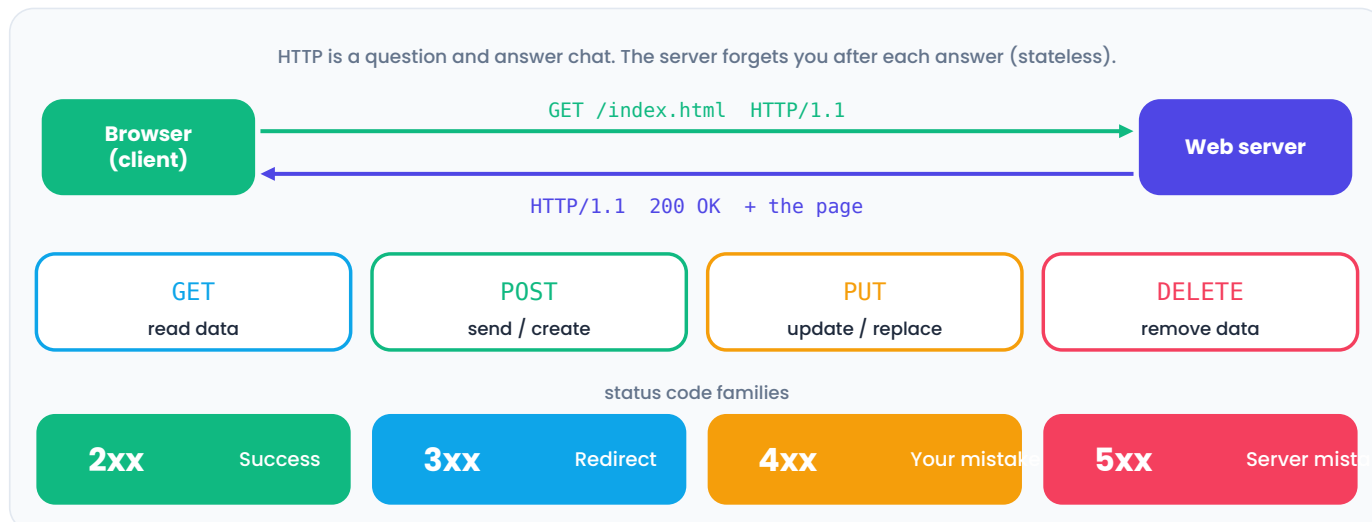


Fig 25. An HTTP request and response, the main methods, and the status code families.

HTTP methods (the verbs)

Method	Meaning
GET	Get some data from the server
POST	Post some form / data to the server
PUT	Put (update) some data
DELETE	Delete some data on the server

Status codes

A **status code** is a short number the server sends back to say how the request went. There are 4 categories: **200s** successful, **300s** redirects, **400s** client errors, **500s** server errors.

Code	Name	What it means
200	OK	Request succeeded
301, 308	Moved Permanently / Permanent Redirect	The address of the resource changed permanently
302, 307	Found / Temporary Redirect	The address of the resource changed temporarily
400	Bad Request	The server cannot understand the request
401	Unauthorized	The client must log in (authenticate) first
403	Forbidden	Logged in, but no permission to access this
404	Not Found	The server cannot find the requested resource
408	Request Timeout	The connection sat idle and the server closes it
500	Internal Server Error	The server does not know how to handle the request (a bug)
502	Bad Gateway	A gateway / reverse proxy got a bad answer from the real server behind it
503	Service Unavailable	The server is down or overloaded
504	Gateway Timeout	Like 502, but the gateway got no answer in time

WHY DEVOPS CARES

4xx usually means the request is wrong (check URL, login, permissions). **5xx** means the server side is broken (check logs, the app, the load balancer). This one rule solves half of all web incidents.

Cookies: how a stateless site remembers you

If HTTP forgets you after every request, how does a shopping site keep you logged in and keep your cart? With **cookies**.

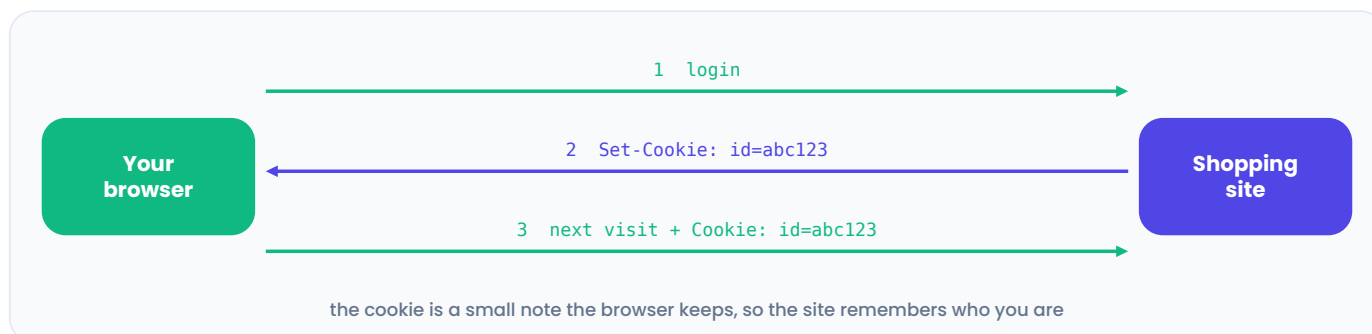


Fig 26. The browser keeps a small note (cookie) and sends it back each time.

KEY IDEA

A **cookie** is a small unique string stored as a file in your browser. On your next visit the browser sends it to the server through HTTP, the server checks it, and automatically logs you in.

HTTPS: HTTP with a lock

Plain HTTP (port 80) sends text anyone on the path can read. **HTTPS (port 443)** wraps HTTP in **TLS** (the successor of SSL) so the data is **encrypted** and the server's identity is checked with a **certificate**.

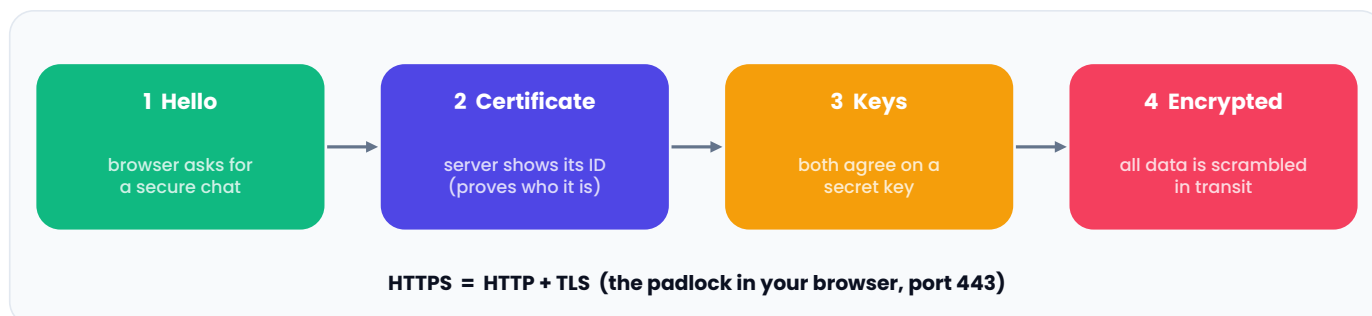


Fig 27. How a secure connection is set up.

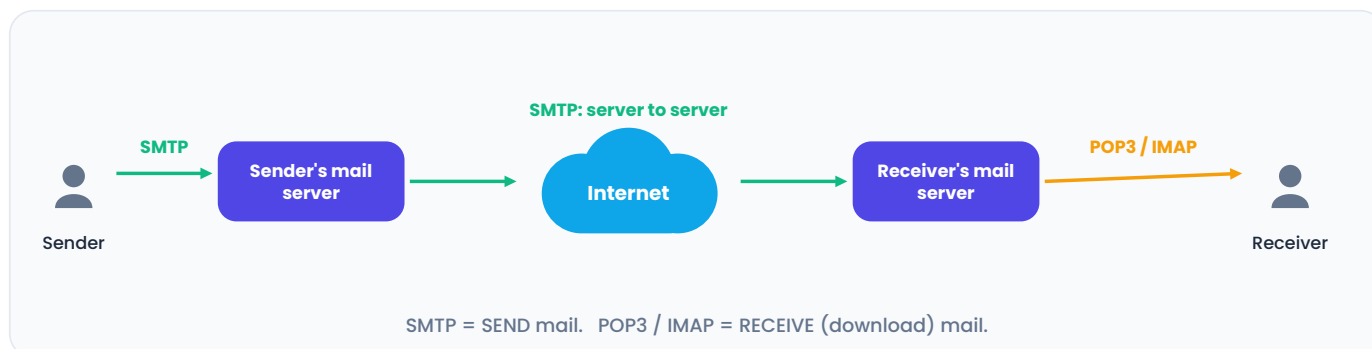
Email: SMTP, POP3 and IMAP

Fig 28. SMTP sends. POP3 / IMAP receive.

Protocol	Port	Job
SMTP	25	Sends email from the sender to the sender's mail server, then on to the receiver's mail server

Protocol	Port	Job
POP3	110	Downloads email from your mail server
IMAP	143	Reads email on the server and keeps it in sync across devices

FTP and SSH

Protocol	Port	What it is for
FTP (File Transfer Protocol)	20, 21	Download, upload and transfer files from one host to another. Not encrypted; prefer SFTP (over SSH) today.
SSH (Secure Shell)	22	Used by system administrators to securely log in to a computer over an insecure network. Similar to Telnet but encrypted .

TCP: the careful courier

- **Reliable** and **connection-oriented**.
- Makes sure data arrives, using **acknowledgements** and **retransmission** if something is lost.
- Used where data must be correct. Example: **HTTP** (port 80), **SQL Server** (port 1433).

The three-way handshake

Before sending data, TCP checks that a stable connection exists between sender and receiver. This is the **three-way handshake**: **SYN**, then **SYN-ACK**, then **ACK**.

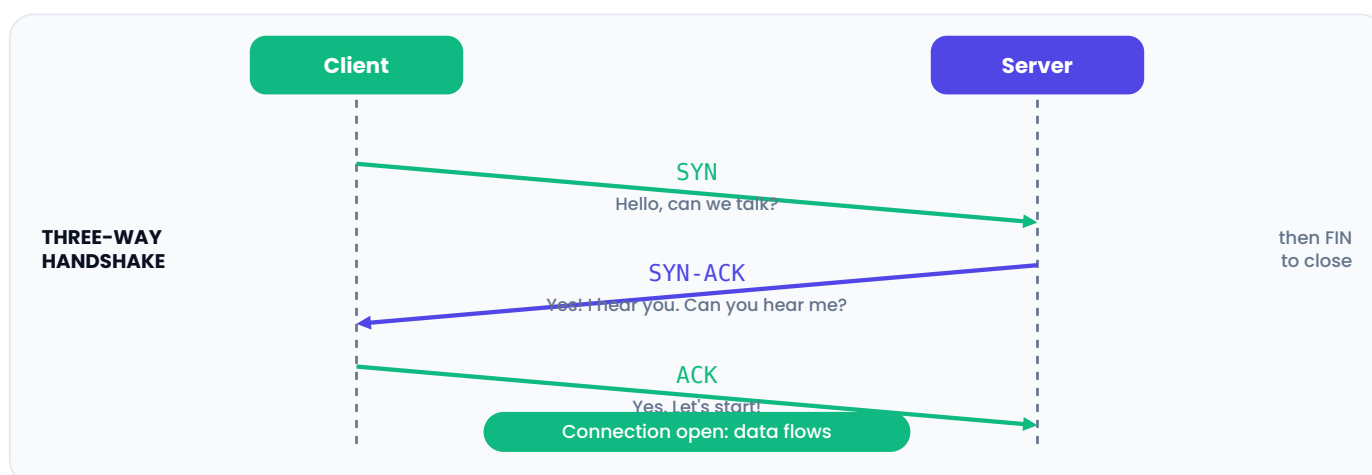


Fig 29. SYN, SYN-ACK, ACK: "Hello?", "Hello, I hear you", "Great, go ahead".

UDP: the fast postcard

- **Unreliable** and **connectionless**.
- No acknowledgements and no retransmission: **faster**, but delivery is not guaranteed.
- Used when **speed matters more than perfection**. Examples: **video streaming** and **DNS** queries.

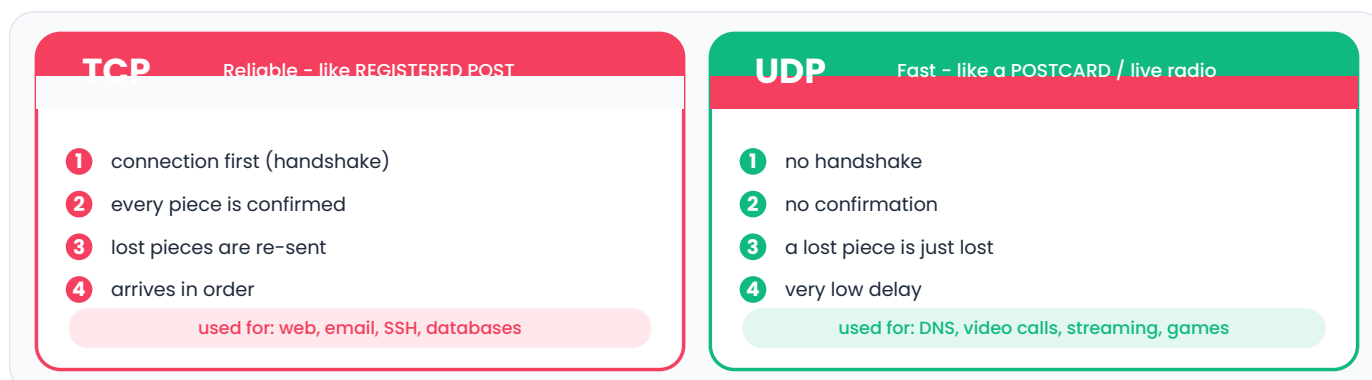


Fig 30. TCP vs UDP.

KEY IDEA

ICMP: the "are you there?" protocol. ICMP sends small control and error messages. The **ping** command uses it (ICMP echo request and echo reply) to test if a host is reachable.

11

Routing, NAT & ARP

How packets find their way



Routing: finding the way

How does a packet get from a host on one network to a host on another? In one word: **routing**. A **router** uses a **route table** to choose the next hop.

THINK OF IT LIKE

Road signs at a junction. The sign says: "Local shops: straight. Other cities: take the highway. Everywhere else: take the airport road." The router checks the signs for each packet.

A typical route table (this one is from the cloud, AWS):

Destination	Target
10.21.0.0/16	local
10.0.0.0/8	a transit gateway (tgw-...)
0.0.0.0/0	a NAT gateway (nat-...)

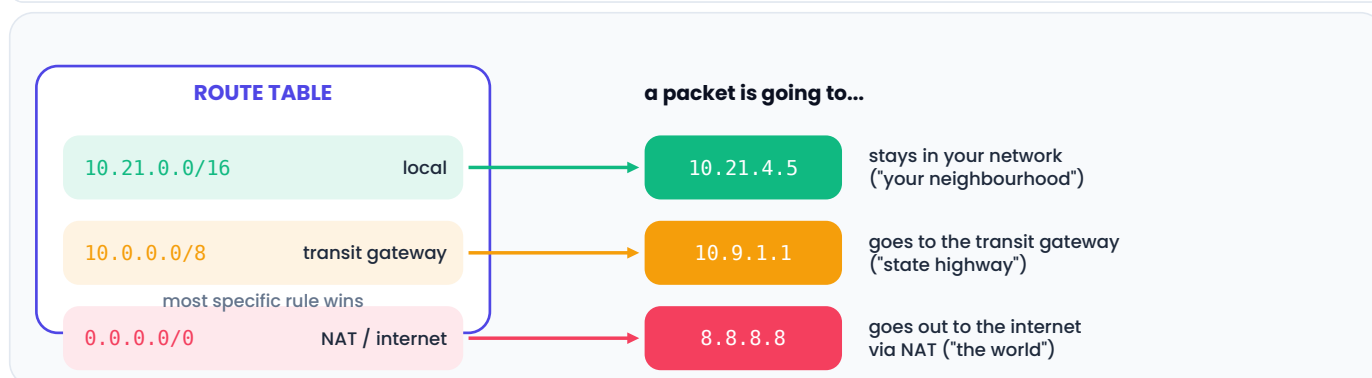


Fig 31. The router picks the most specific matching rule.

When making a routing decision, the narrowest (most specific) rule is checked first.

- Destination inside **10.21.0.0/16**: stays in the local network (your neighbourhood).
- Destination inside **10.0.0.0/8**: sent to the transit gateway (your state highway).
- Anything else matches the widest rule **0.0.0.0/0** = internet traffic, sent to the **NAT** interface.

Default gateway

The **default gateway** is the router a device uses when the destination is **not** in its own network. It is the door out of your LAN. The route **0.0.0.0/0** is the "default route" that points to it.

Static and dynamic routing

Kind	How routes are learned	Examples
Static	An admin types the routes by hand. Simple, predictable.	Small networks, cloud route tables
Dynamic	Routers share routes with each other automatically.	RIP, OSPF (inside an organization), BGP (between organizations, runs the Internet)

NAT: many private devices, one public IP

Private addresses cannot go on the internet. **NAT (Network Address Translation)** lets them share one public IP. The router swaps the private source address for its public one, remembers the conversation in a table, and sends replies back to the right device.

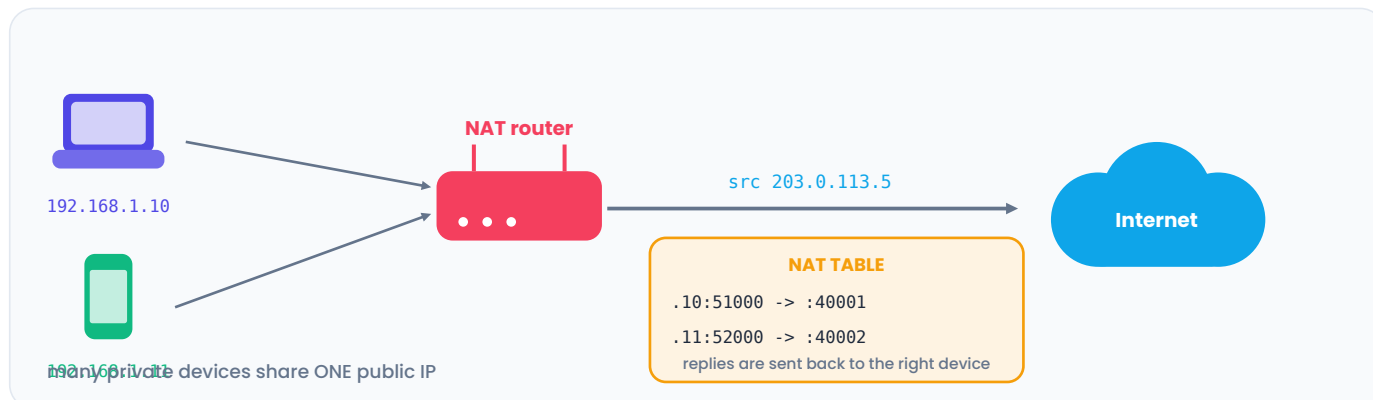


Fig 32. NAT keeps a table so replies find the right device.

EXAMPLE

A company switchboard. Everyone has an internal extension (private IP), but outside callers only see one main phone number (public IP). The receptionist (NAT) connects each call to the right extension.

WHY DEVOPS CARES

In the cloud a **NAT gateway** lets servers in a **private subnet** download updates, while the internet still cannot start a connection to them (see Section 14).

ARP: finding a MAC from an IP

On a local network, delivery uses MAC addresses, but you usually only know the IP. **ARP (Address Resolution Protocol)** asks the whole LAN "who has this IP?" and learns the matching MAC address.

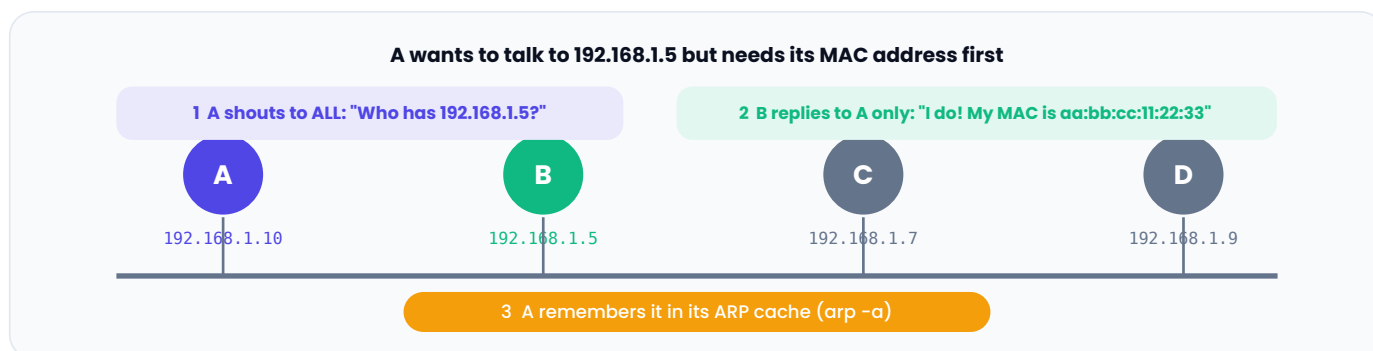


Fig 33. ARP in three steps.

12

DNS

The phonebook of the internet

Computers talk using IP addresses, but humans remember names. **DNS (Domain Name System)** translates a human-readable name like **www.example.com** into an IP address like **192.0.2.44**.

THINK OF IT LIKE

The phonebook of the internet. You look up a friend's name and get their number. You type a website name and DNS gives your browser the number (IP address) to call.

What happens when you visit a website

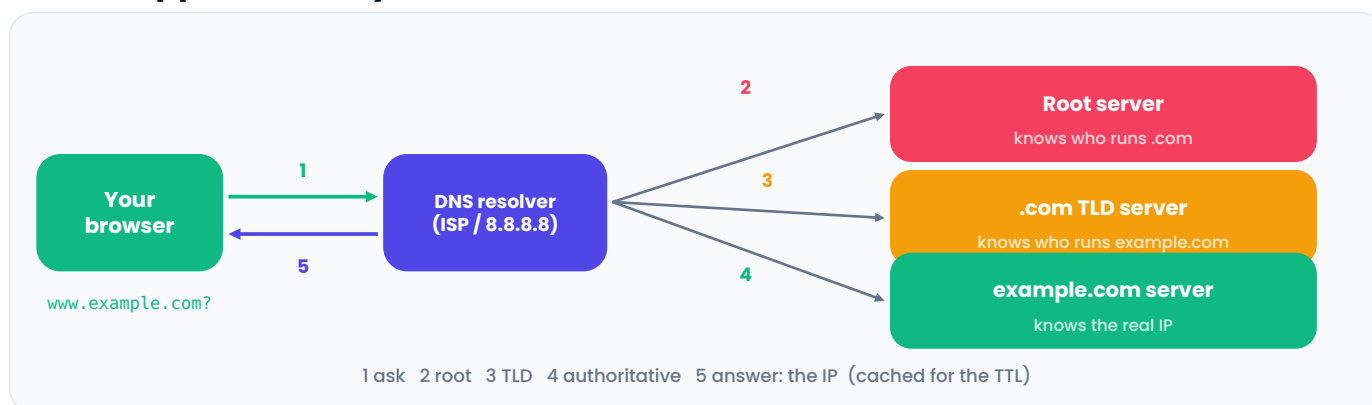


Fig 34. How a name becomes an IP address.

1. **User request:** you type **www.example.com** in your browser.
2. **DNS query:** your browser asks a DNS server (the resolver) for the IP address of that name.
3. **DNS resolution:** the resolver checks if it already has the answer saved (**cached**). If not, it asks other DNS servers: **root**, then **TLD**, then the **authoritative** server.
4. **IP address found:** for example **93.184.216.34**. The resolver sends it back to your browser.
5. **Connecting:** the browser uses that IP to reach the web server, and the page loads.

Domains, zones and delegation

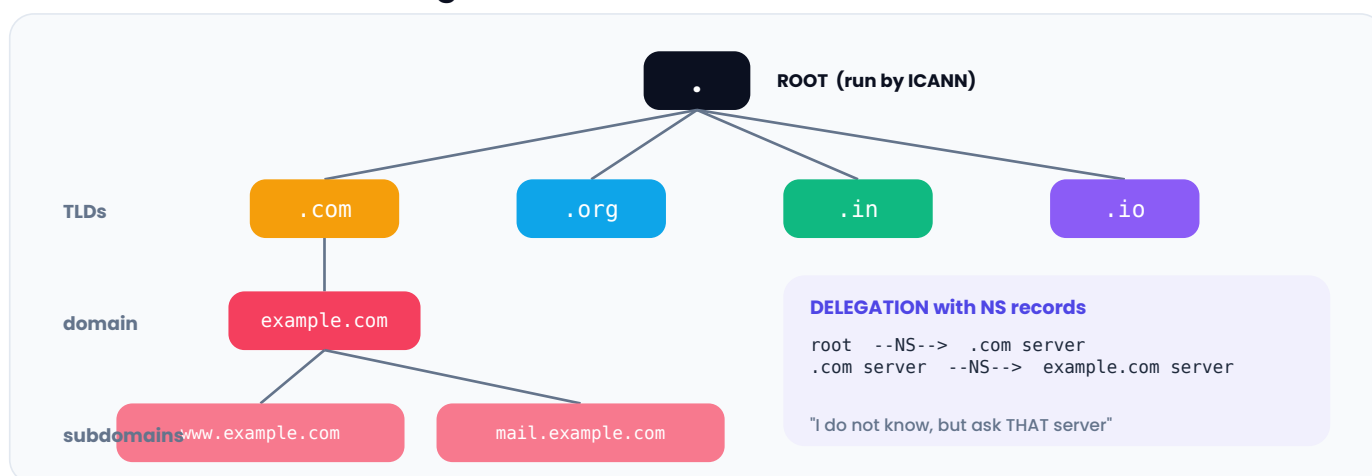


Fig 35. DNS is a tree. Each level can be managed by a different organization.

1. **Domains:** branches of a tree. The **root** is the top, then **top-level domains (TLDs)** such as **.com**, **.org**, **.in**, **.io**, then names like **example.com** and subdomains like **www.example.com**. The root servers know which servers handle each TLD.

2. **Zones:** a zone is a part of the tree managed by one organization. For instance `.com` is a zone run by **Verisign**, and **ICANN** manages the root zone at the top.
3. **Delegation:** one organization hands control of part of its domain to another using **Nameserver (NS) records**. ICANN delegates `.com` to Verisign; Verisign delegates `example.com` to "Example Ltd" with an NS record that points to their nameserver.

So **NS records** direct traffic to the right nameserver, which lets different parts of the DNS tree be managed independently.

DNS record types

DNS records (also called zone files) tell DNS what to answer for a domain. Each record has a **TTL (time to live)**: how long, in seconds, a DNS server may cache it before asking again.

Type	Name	What it does
A	Host address	The most basic and common record. Turns a name into an IPv4 address.
AAAA	IPv6 host address	Same as A, but for IPv6 .
CNAME	Canonical name (alias)	Maps a name to another name . Use it only when there are no other records on that name.
ALIAS	Auto-resolved alias	Maps a name to another name but can coexist with other records on that name.
MX	Mail eXchange	Says which mail servers handle email for the domain.
NS	Name Server	Says which DNS servers are responsible for the zone (one record per server).
TXT	Descriptive text	General information about the domain. Widely used for domain ownership verification and email security.
PTR	Pointer	The reverse of A: turns an IP address back into a name.

Try DNS yourself

Command	What it does
<code>dig example.com</code>	Full DNS answer: IP, TTL, record type
<code>dig example.com +short</code>	Only the answer
<code>dig MX example.com</code>	Ask for the mail servers
<code>nslookup example.com</code>	Simple lookup (also on Windows)

Sample:

```
$ dig example.com +short
93.184.216.34
```

WHY DEVOPS CARES

"It is always DNS." A huge share of outages are DNS problems: a wrong A record, a missing record, or an old cached answer (high TTL) after a change. Before a planned server move, **lower the TTL** so the change spreads quickly. A computer also checks its own file first: **/etc/hosts** (Linux) maps names to IPs by hand.

13

DHCP & Network Services

DHCP, firewall, load balancer, VPN, VLAN

DHCP: automatic IP addresses

DHCP (Dynamic Host Configuration Protocol) automatically gives a device its **IP address** and other settings (**subnet mask, default gateway, DNS servers**) when it joins a network, so nobody has to type them in.

EXAMPLE

You connect your laptop to a Wi-Fi network. A **DHCP server** (usually the router) assigns it an IP address automatically, so it can talk to other devices without any manual setup.



Fig 36. DHCP in four steps (DORA).

Step	Message	In plain words
D	Discover	The new device shouts: "Is there a DHCP server?"
O	Offer	The server offers an address
R	Request	The device says: "I will take it"
A	Acknowledge	The server confirms. The address is leased for a time (the lease)

If DHCP fails, a device may give itself an address starting with **169.254**. Seeing one is a clue that DHCP is not working.

Network services you will meet in DevOps

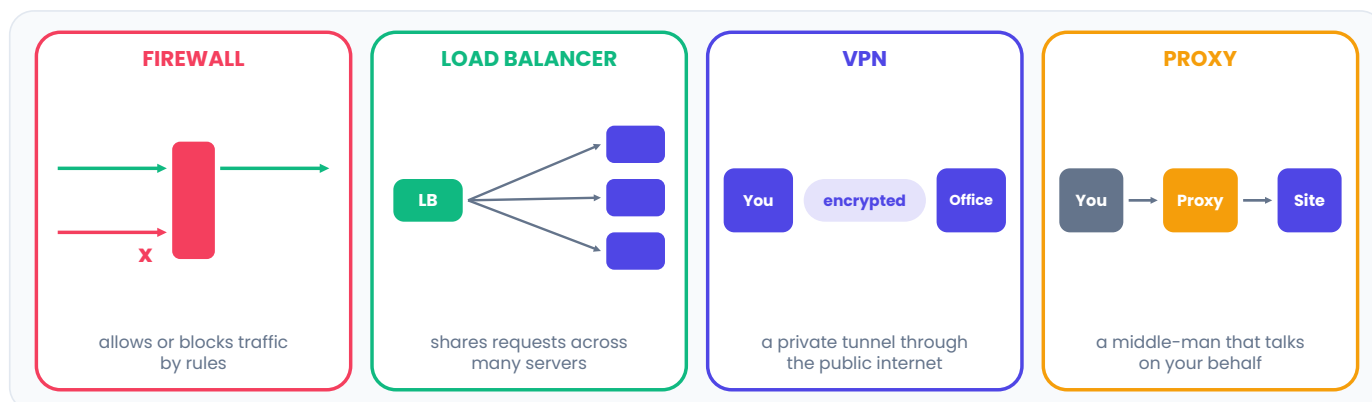


Fig 37. Four network helpers.

Service	What it does	Example
Router	Connects different networks and directs packets between them	Home router, cloud router
Switch	Connects devices inside the same network , using MAC addresses	Office switch

Service	What it does	Example
Firewall	Controls incoming and outgoing traffic based on predetermined security rules	iptables, ufw, cloud security groups
Load balancer	Distributes incoming traffic across multiple servers so no single server is overwhelmed	Nginx, HAProxy, cloud load balancers
VPN	Virtual Private Network: a secure connection between remote users and the corporate network over the internet	WireGuard, OpenVPN
Proxy	A middle-man. A reverse proxy sits in front of servers, receives requests and passes them on	Nginx, Envoy

Firewall rules in practice

A firewall reads its rules from top to bottom. A good starting point is **block everything, then allow only what you need**.

Rule	Protocol / port	From	Action
SSH for admins	TCP 22	Your office IP only (203.0.113.7/32)	ALLOW
Website	TCP 80, 443	Anywhere (0.0.0.0/0)	ALLOW
Database	TCP 3306	Only the app servers' subnet	ALLOW
Everything else	any	any	DENY

WATCH OUT

Never open SSH (22) or a database port to **0.0.0.0/0**. Bots scan the whole internet within minutes of a server going online.

Load balancer in one minute

A load balancer is the single front door. Users talk to one address; it sends each request to a healthy server and stops using a server that fails its health check. **Layer 4** balancers look at IP and port only (fast). **Layer 7** balancers read HTTP (URLs, headers, cookies) and can route **/api** to one group and **/images** to another.

VLAN: virtual networks on one switch

A **VLAN** splits one physical switch into several separate virtual networks, so Office and Guest devices cannot see each other even though they plug into the same box.

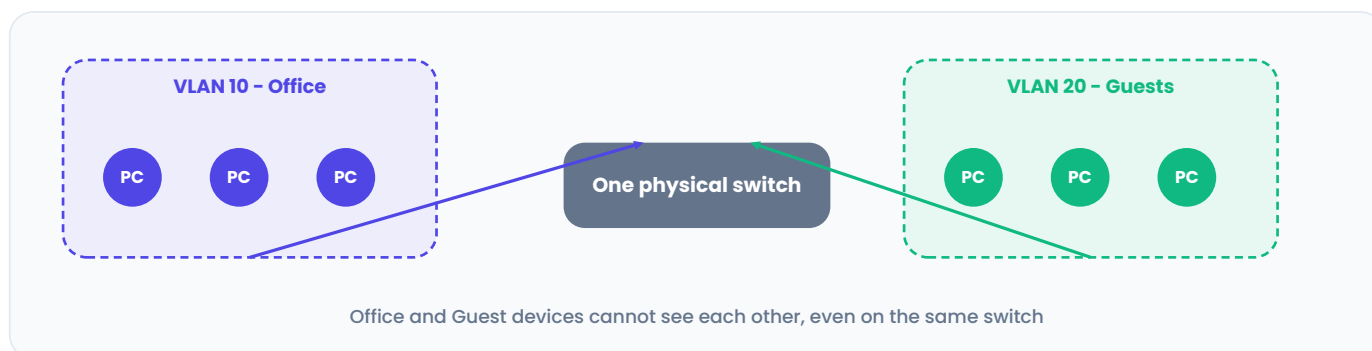


Fig 38. Two VLANs on one switch.

Basic network security habits

- **Allow only what you need** (least privilege) and close unused ports.
- **Encrypt traffic:** use HTTPS, SSH and VPN instead of plain HTTP, Telnet and FTP.
- **Separate networks:** keep databases in private subnets, away from the internet.
- **Keep software updated** and log who connects.

14

Cloud & Container Networking

VPC, Docker and Kubernetes networking

Today most servers live in the cloud or in containers. The ideas you just learned (IP, subnets, routes, ports, firewalls) are exactly what you configure there, only with a web console or code.

Cloud networking: the VPC (example: AWS)

A **VPC (Virtual Private Cloud)** is your own private network inside the cloud, like having your own data centre. You choose its IP range (for example **10.0.0.0/16**) and split it into **subnets**.

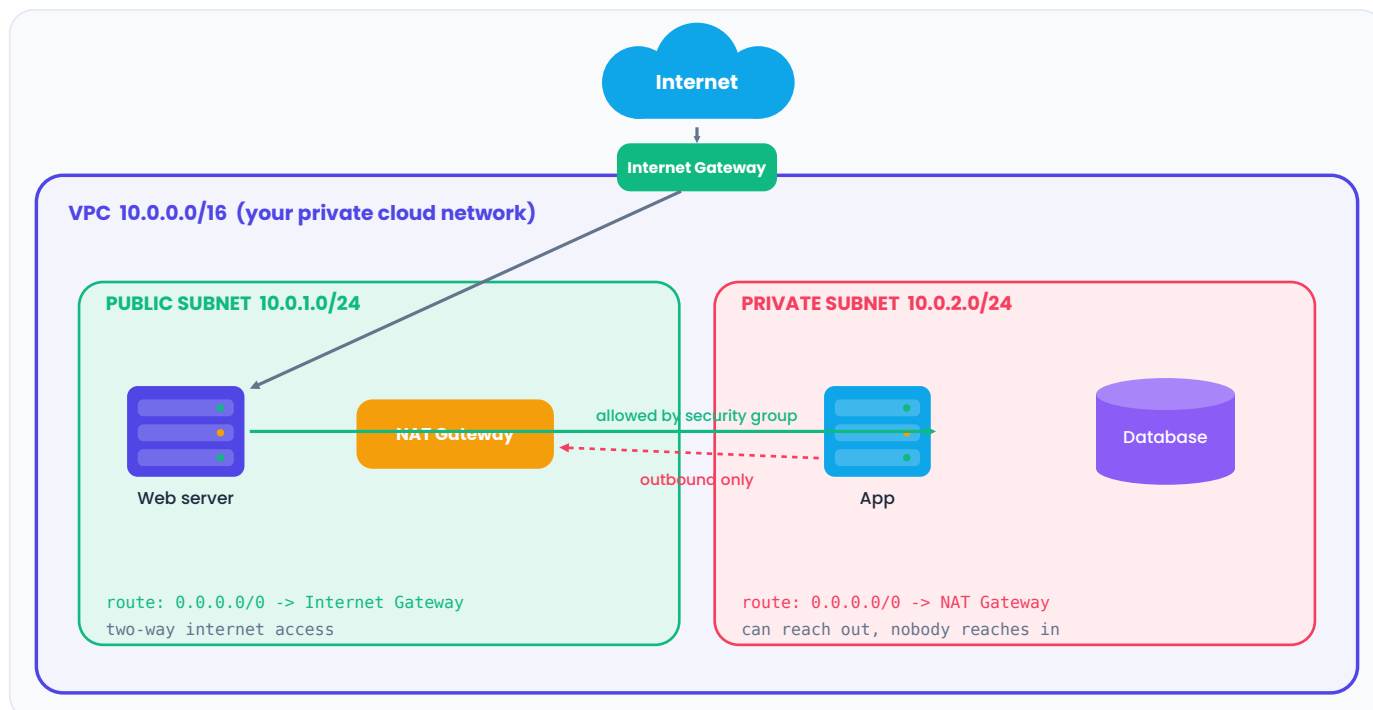


Fig 39. A typical VPC with a public and a private subnet.

Part	What it is	Plain meaning
VPC	Your isolated network in one cloud region	Your own fenced plot of land
Subnet	A range of IPs inside the VPC (public or private)	A room in your building
Route table	Rules that say where traffic goes	The road signs
Internet Gateway (IGW)	Lets the VPC talk to the internet, both ways	The main gate
NAT Gateway	Lets private servers reach the internet outbound only , no one can reach them first	A one-way door
Security Group	A firewall on each server. Only allow rules. Stateful.	A bouncer at each door
Network ACL	A firewall on the whole subnet. Stateless: allow and deny rules, checked both ways.	The guard at the corridor entrance

Public vs private subnet

	Public subnet	Private subnet
Route to the internet	0.0.0.0/0 -> Internet Gateway	0.0.0.0/0 -> NAT Gateway (or no internet route)
Reachable from internet?	Yes (if it has a public IP and the firewall allows)	No
Typical resources	Load balancer, web server, NAT gateway	App servers, databases
Rule of thumb	A subnet is public when its route table points to an Internet Gateway	Hidden from the internet. Use NAT or a VPN for outbound access

KEY IDEA

Ask one question to choose: **"Should this server be reachable directly from the internet?"** Yes = public subnet. No = private subnet (databases almost always go here).

Security group vs network ACL

	Security Group	Network ACL
Works on	A server / network interface	A whole subnet
Rule types	Allow only	Allow and deny
State	Stateful : replies are allowed automatically	Stateless : you must allow the return traffic too
Default	Everything inbound denied until you add rules	Default one allows all

Docker networking

A container gets its own private network address. By default Docker creates a **bridge** network on the host: a virtual switch (**docker0**, usually 172.17.0.1) that every container plugs into. To let the outside world in, you **publish a port**.

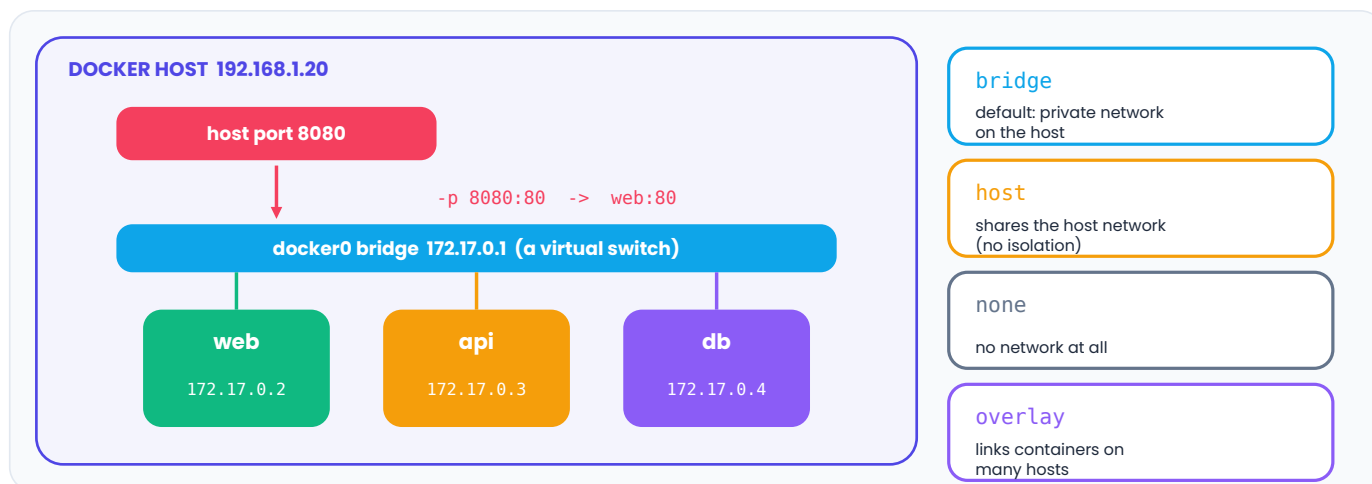


Fig 40. Docker bridge network and the four network drivers.

Driver	What it does
bridge	The default. Containers on the same host can talk to each other; the outside is blocked unless you publish a port.
host	The container uses the host's network directly (no isolation, Linux only).
none	No network at all: fully isolated.
overlay	Joins containers across many hosts (Docker Swarm / Kubernetes).

```

docker network ls           # list networks
docker run -d -p 8080:80 nginx # host port 8080 -> container port 80
docker run --rm --network host nginx # use the host network
docker network create my-net # your own bridge network

```

Kubernetes networking

In Kubernetes every **Pod** gets its **own IP address**. But Pods are created and destroyed all the time, so their IPs keep changing. A **Service** gives a group of Pods one **stable IP and name** and balances traffic across them. An **Ingress** receives traffic from outside and routes it by domain name or path.

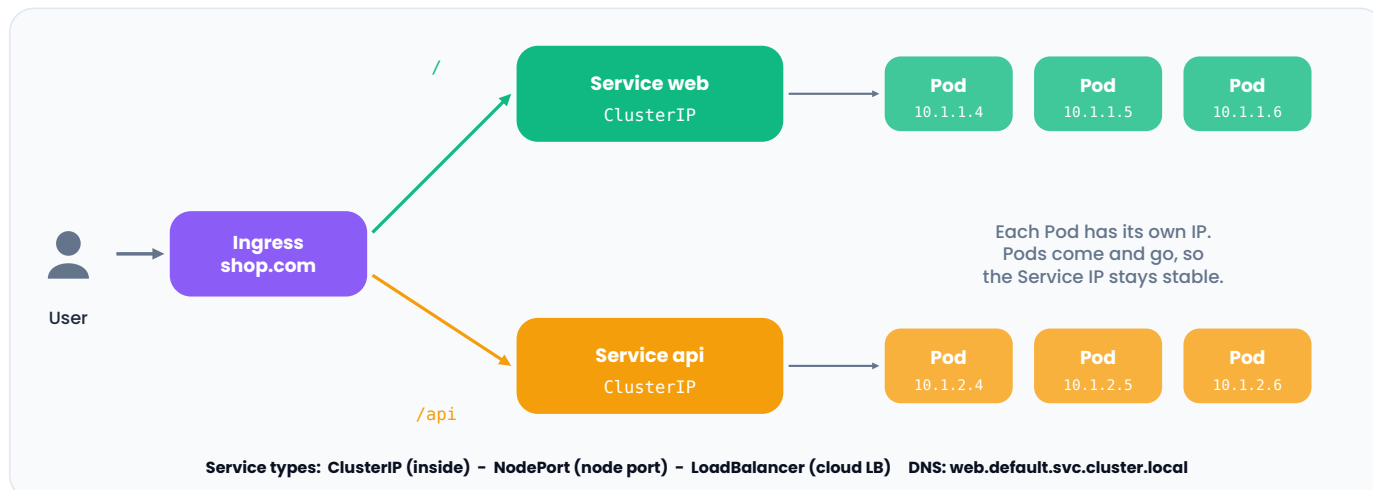


Fig 41. User > Ingress > Service > Pods.

Service type	What it does	Use it for
ClusterIP (default)	A virtual IP reachable only inside the cluster	Internal services, databases
NodePort	Opens the same port on every node's IP	Quick tests, simple setups
LoadBalancer	Asks the cloud to create an external load balancer with a public IP	Exposing an app to the internet
Ingress	Smart HTTP routing by host / path in front of Services	Many websites on one entry point

Kubernetes has a built-in **DNS**. Pods reach a Service by name, for example **web.default.svc.cluster.local** (service.namespace.svc.cluster.local), instead of hard-coding IP addresses.

WHY DEVOPS CARES

Cloud and containers did not change networking, they **wrapped** it. When a pod cannot reach a database, you still ask the same questions: right IP? route? port open? firewall / security group? DNS resolving?

15

Troubleshooting Tools

ping, traceroute, curl, dig and more



When something does not work, do not guess. Follow the path of the packet and test one step at a time.

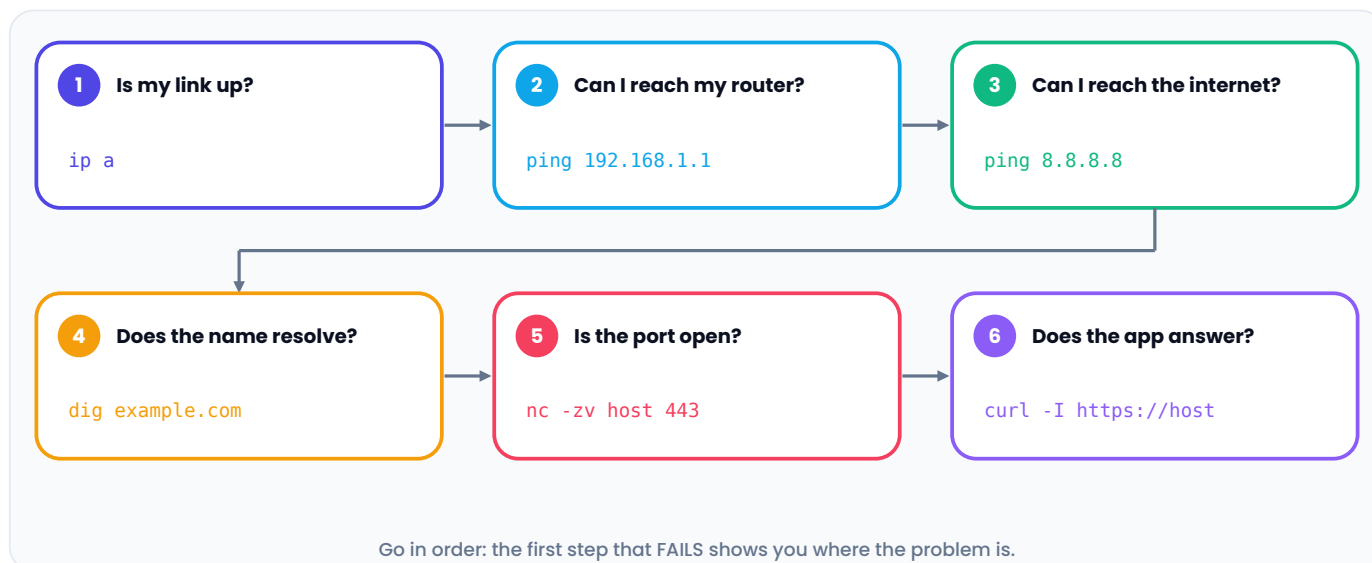


Fig 42. A simple troubleshooting path. The first step that fails shows the problem.

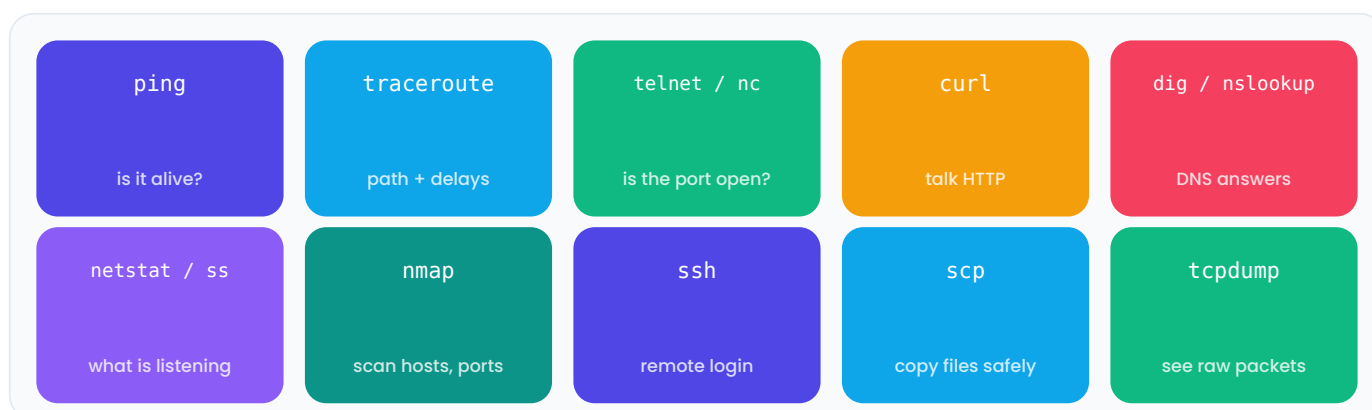


Fig 43. Your toolbox.

1. ping

Purpose: test if a host is reachable and how long it takes.

How it works: sends an **ICMP ECHO_REQUEST** and waits for an **ICMP ECHO_RESPONSE** from the remote host.

Command	What it does
<code>ping www.google.com</code>	Check network connectivity, response times and packet loss
<code>ping -c 4 8.8.8.8</code>	Send only 4 pings (Linux / macOS)

```

$ ping -c 3 example.com
64 bytes from 93.184.216.34: icmp_seq=1 ttl=56 time=11.8 ms
64 bytes from 93.184.216.34: icmp_seq=2 ttl=56 time=11.6 ms
64 bytes from 93.184.216.34: icmp_seq=3 ttl=56 time=11.9 ms
3 packets transmitted, 3 received, 0% packet loss
  
```

time= is the latency. **0% packet loss** means every packet came back. Some servers block ping on purpose, so no reply does not always mean "down".

2. traceroute (tracert on Windows)

Purpose: show the route packets take to the destination.

How it works: sends probes with **increasing TTL** values, so each router along the way reports back. It helps find which gateway adds delay, and where packets stop (shown as * * *).

Command	What it does
<code>traceroute example.com</code>	List every router (hop) on the way and the delay to reach it
<code>tracert example.com</code>	The same on Windows

```
$ traceroute example.com
1  192.168.1.1      1.2 ms    <- your router
2  10.20.0.1       8.5 ms    <- your ISP
3  * * *           <- no reply here
4  93.184.216.34   22.1 ms   <- destination
```

3. telnet and nc

Purpose: test if a specific **port** on a host can be reached.

How it works: attempts to establish a connection to the given host and port. "Connected" means the service is reachable.

Command	What it does
<code>telnet google.com 443</code>	Try to open a connection to that IP / name and port
<code>nc -zv google.com 443</code>	Same test with netcat (the "-z" only checks, "-v" is verbose)

4. curl

Purpose: transfer data using many protocols, mostly HTTP requests.

Command	What it does
<code>curl http://example.com</code>	Basic GET request
<code>curl -I http://example.com</code>	Check only the headers (status code)
<code>curl -X POST http://example.com</code>	Send a POST request
<code>curl http://example.com/file -o output.file</code>	Save the response to a file
<code>curl -v https://example.com</code>	Verbose: see the whole conversation

5. dig (Domain Information Groper) and nslookup

Purpose: troubleshoot **DNS** problems and verify DNS records.

How it works: performs DNS lookups and displays the answer.

Command	What it does
<code>dig google.com</code>	IP addresses, TTL and DNS record types
<code>dig MX google.com</code>	Ask for a specific record type
<code>nslookup google.com</code>	Simple lookup

6. netstat and ss

Purpose: show network connections and which ports are **listening**. **ss** is the modern replacement for netstat.

Command	What it does
<code>netstat -lp</code>	List listening servers and their program names
<code>netstat -a</code>	Show all active ports
<code>netstat -r</code>	Show the routing table
<code>ss -tuln</code>	Listening TCP / UDP ports, numeric (modern)

7. nmap (Network Mapper)

Purpose: discover hosts and services on a network.

How it works: sends raw packets to identify hosts, services and operating systems.

Command	What it does
<code>nmap -sn 172.31.44.35/20</code>	Discover which hosts are alive in a subnet
<code>nmap -A 172.31.36.237</code>	Scan the ports of one host (and guess services and OS)

WATCH OUT

Only scan networks and servers that **you own or have written permission** to test.

8. ssh (Secure Shell)

Purpose: securely connect to a remote machine and run commands. It is encrypted and used for remote management and file transfers.

Command	What it does
<code>ssh username@hostname</code>	Connect to a server
<code>ssh -i key.pem ubuntu@203.0.113.10</code>	Connect using a key file

9. scp (Secure Copy)

Purpose: securely copy files between your computer and a remote host.

Command	What it does
<code>scp localfile.txt user@remote:/path/to/destination</code>	Copy a file to a remote server
<code>scp user@remote:/path/file.txt .</code>	Copy a file from the server to here

10. ip and tcpdump (extras you will love)

Purpose: inspect your own network setup and watch real packets.

Command	What it does
<code>ip a</code>	Show your interfaces and IP addresses
<code>ip route</code>	Show the routing table and default gateway
<code>sudo tcpdump -i eth0 port 80</code>	Watch live HTTP packets on interface eth0

KEY IDEA

These tools are valuable for **network diagnostics, troubleshooting, and secure communication**, which are critical skills for any DevOps engineer.

16

Cheat Sheet & Glossary

Revise fast, learn next



Common errors and what they usually mean

What you see	Usually means	Check
Name or service not known	DNS cannot find the name	<code>dig name</code> , resolver, /etc/hosts
Connection refused	Server reachable, but nothing is listening on that port	Is the service running? Right port? <code>ss -tuln</code>
Connection timed out	No answer at all: firewall dropping, wrong IP, or host down	Security group / firewall, route, <code>ping</code>
No route to host	Your computer has no path to that network	<code>ip route</code> , subnet, gateway
HTTP 502 / 503 / 504	Proxy or load balancer cannot get a good answer from the app	App health, logs, load balancer targets
HTTP 403 / 401	Blocked or not logged in	Permissions, credentials

Numbers to remember

Topic	Remember
Private IP ranges	<code>10.0.0.0/8</code> <code>172.16.0.0/12</code> <code>192.168.0.0/16</code>
Loopback	<code>127.0.0.1</code> (localhost)
Whole internet	<code>0.0.0.0/0</code>
/24, /16, /8	256, 65,536, 16,777,216 addresses
Usable hosts	total addresses - 2
Ports	SSH 22, HTTP 80, HTTPS 443, DNS 53, SMTP 25, MySQL 3306, PostgreSQL 5432
TCP vs UDP	TCP = reliable (web, SSH). UDP = fast (DNS, video)
Handshake	SYN, SYN-ACK, ACK
DHCP	Discover, Offer, Request, Acknowledge

Mini glossary

Term	Meaning	Term	Meaning
Host	Any device with an IP	Packet	A small piece of data with an address
Gateway	The router that leads out of a network	Frame	A packet wrapped with MAC addresses
Subnet	A smaller network inside a bigger one	Octet	8 bits (one number in IPv4)
CIDR	Prefix notation like /24	TTL	How long something may be cached
NAT	Many private IPs share one public IP	Latency	Delay of a trip
DHCP	Hands out IP addresses automatically	Bandwidth	Maximum data per second
DNS	Names to IP addresses	Stateless	Server keeps no memory between requests
VPC	Private network in the cloud	Firewall	Allows or blocks traffic by rules

Your learning roadmap

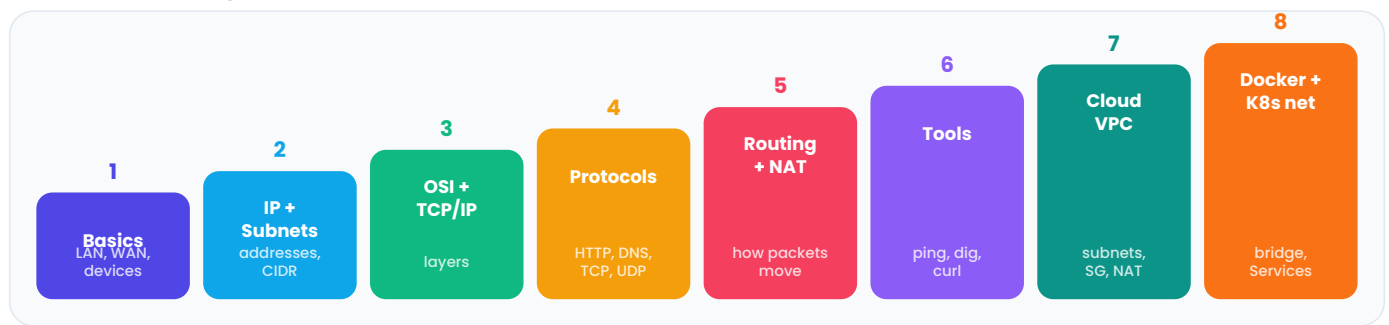


Fig 44. Learn in this order and practise each step with a small lab.

TRY IT

Mini lab: 1) run **ip a** and find your IP, mask and gateway. 2) **ping** your gateway, then 8.8.8.8, then a domain name. 3) **dig** that domain. 4) **curl -I** a website and read the status code. 5) Run **ss -tln** and name each listening port.

KEY IDEA

That is a full tour of networking for DevOps. Every problem you meet will be one of these ideas: an address, a route, a port, a name, or a rule.